

# Week 8

# Transformers

Nebius Academy

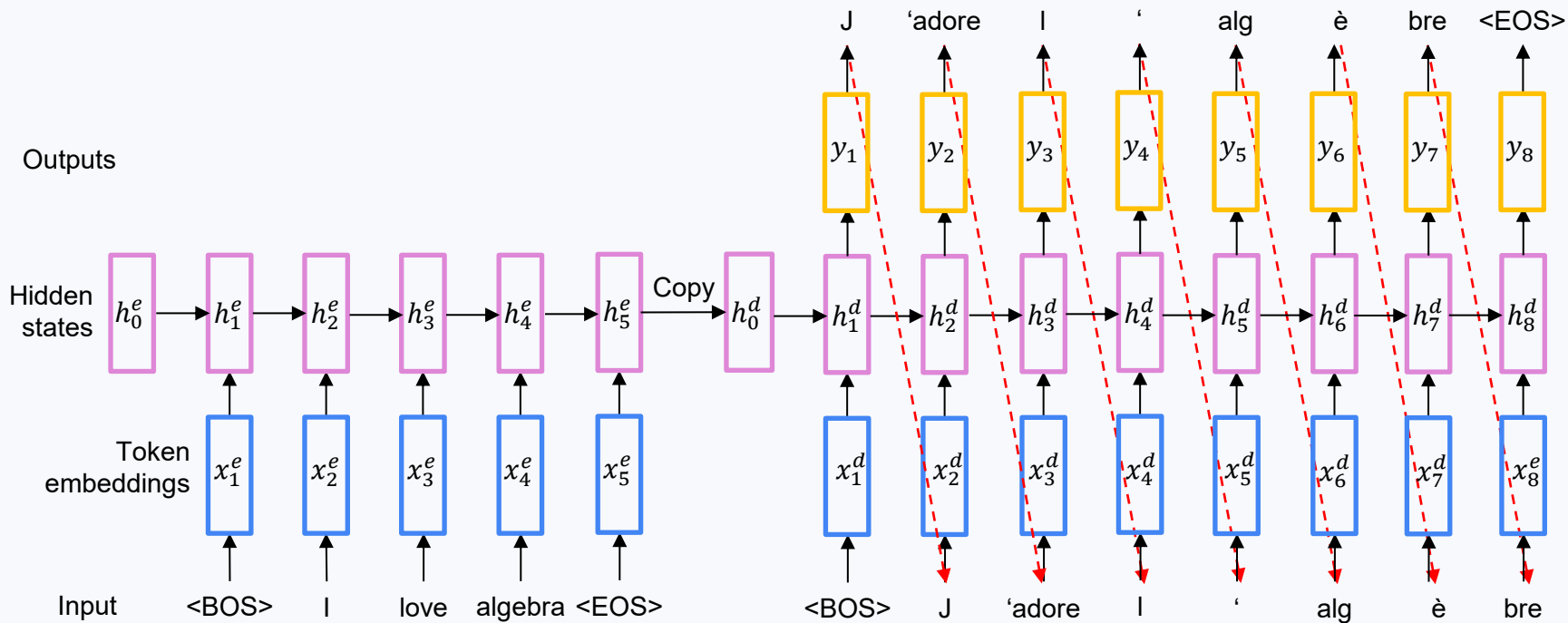


Seq2seq

# Encoder-decoder

## Encoder

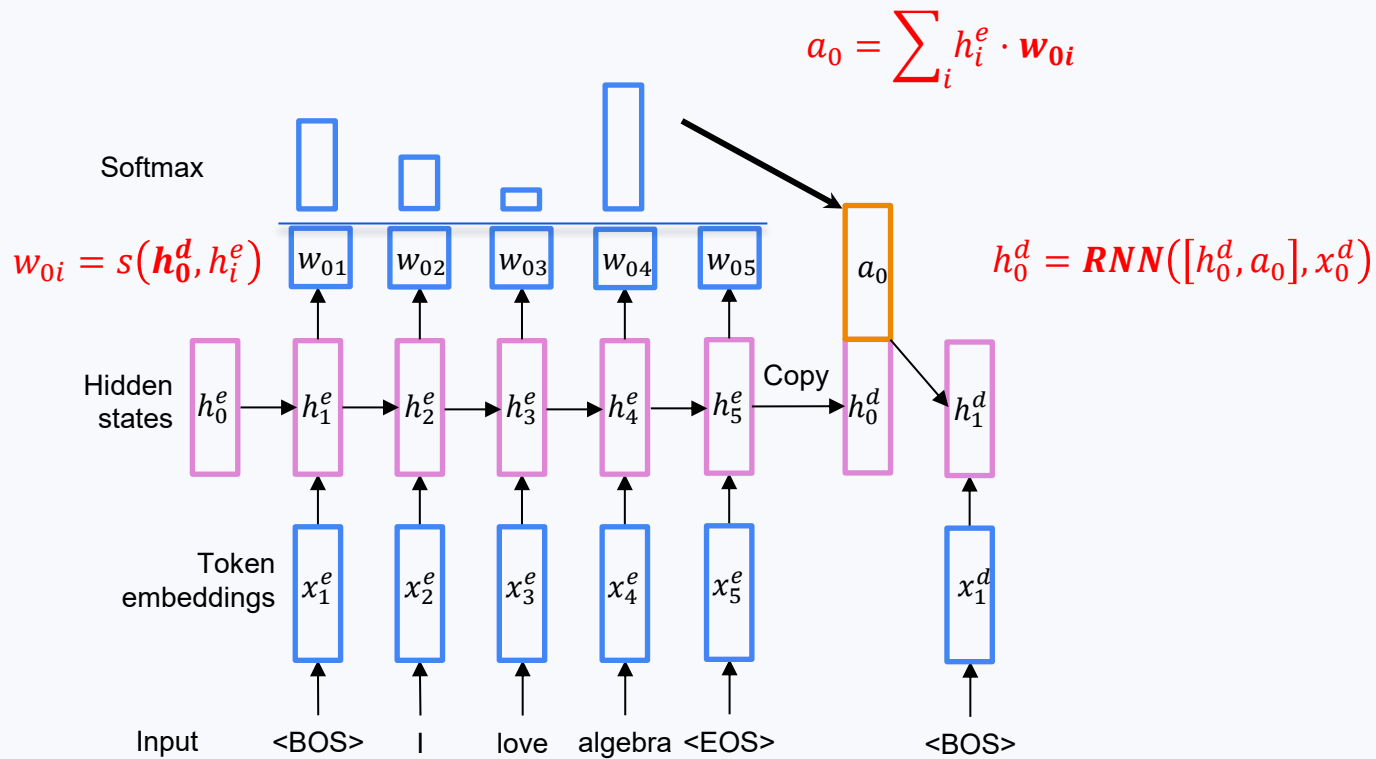
## Decoder



(Bahdanau) attention

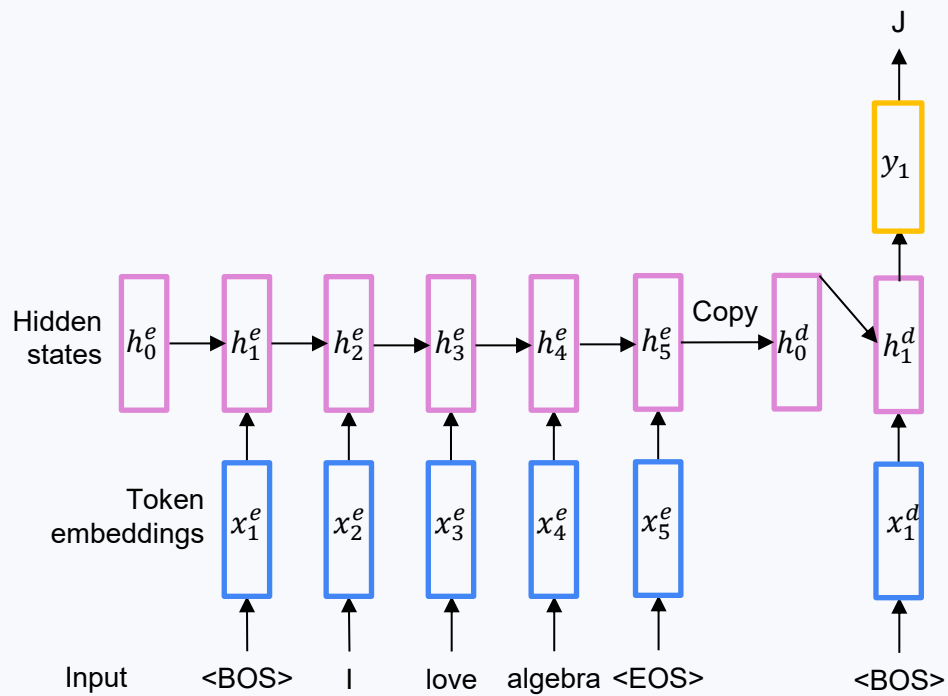
## Encoder

## Decoder



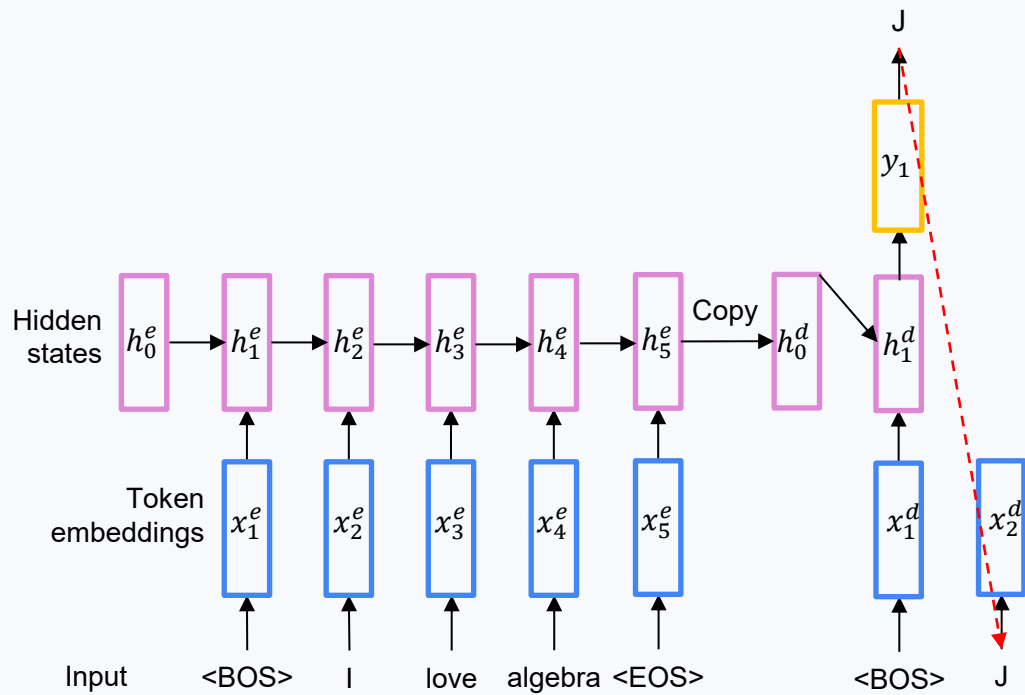
## Encoder

## Decoder

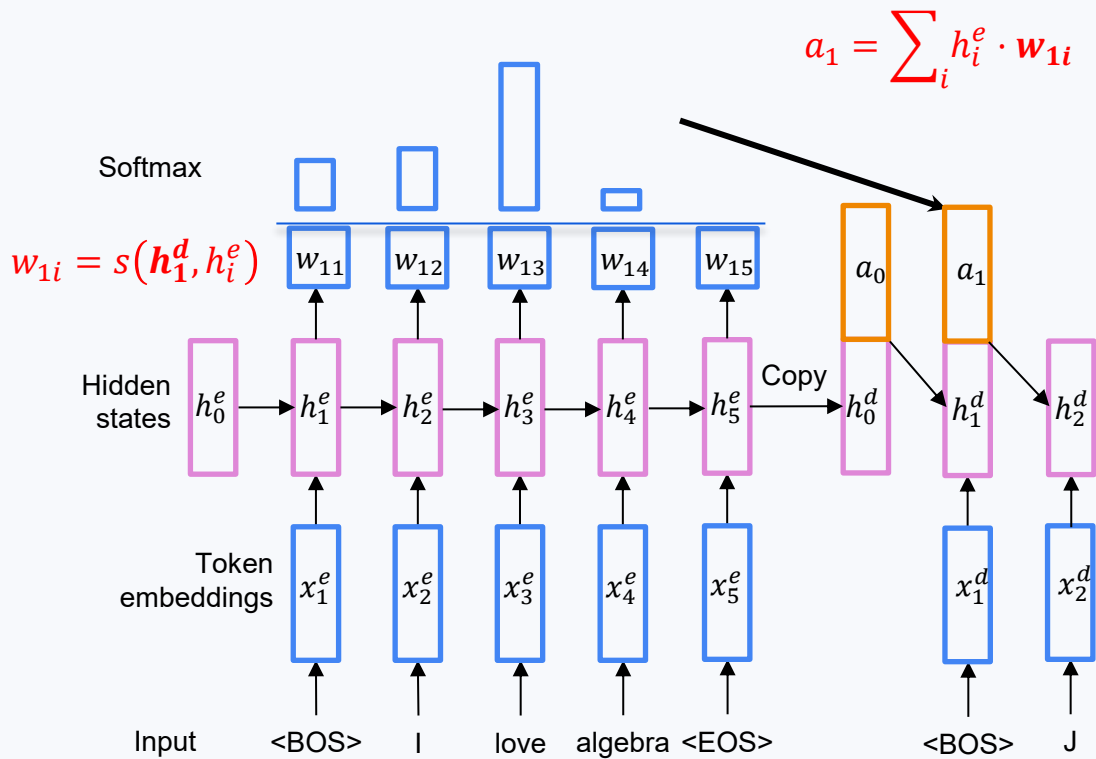


## Encoder

## Decoder



## Encoder



## Decoder

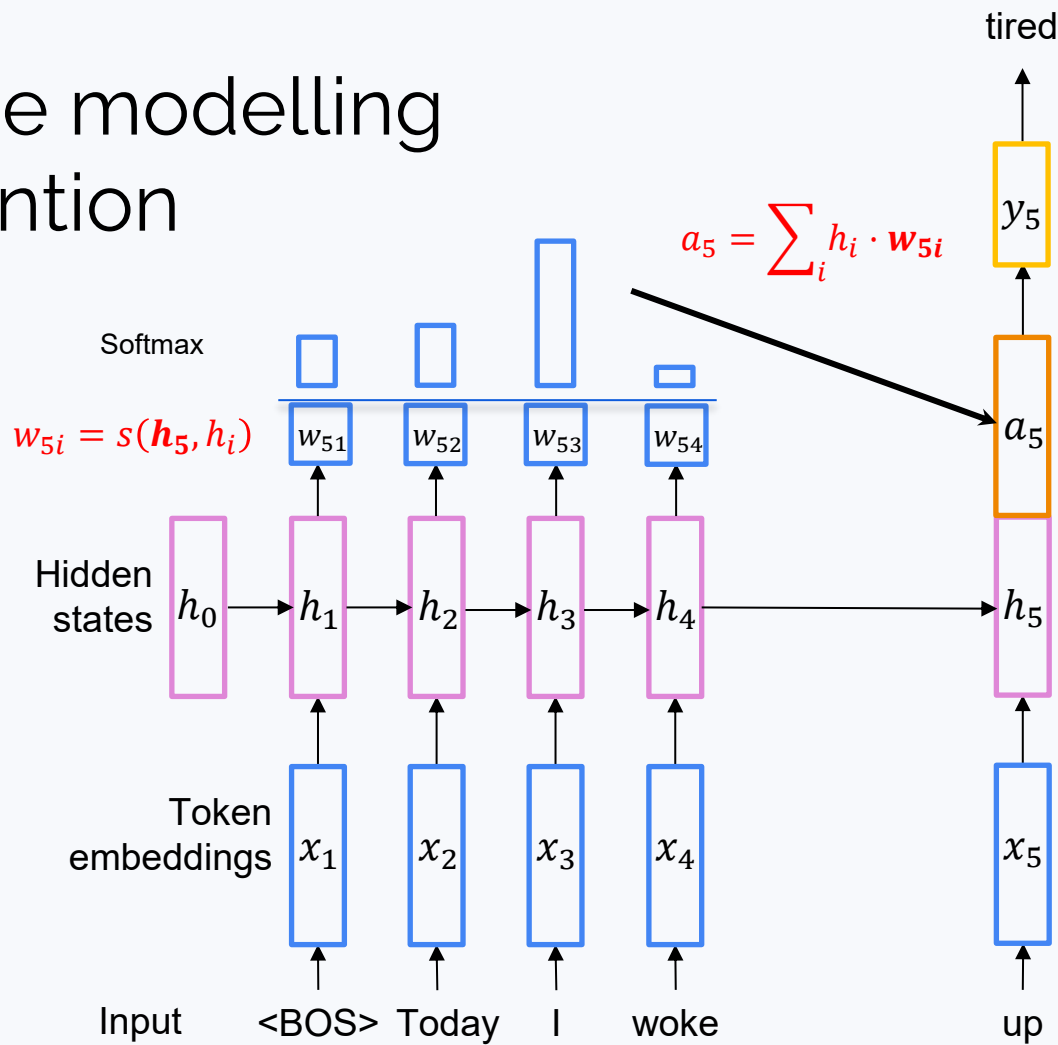
$$h_2^d = RNN([h_1^d, a_1], x_1^d)$$

$s$  is a similarity score  
It might be, for example:

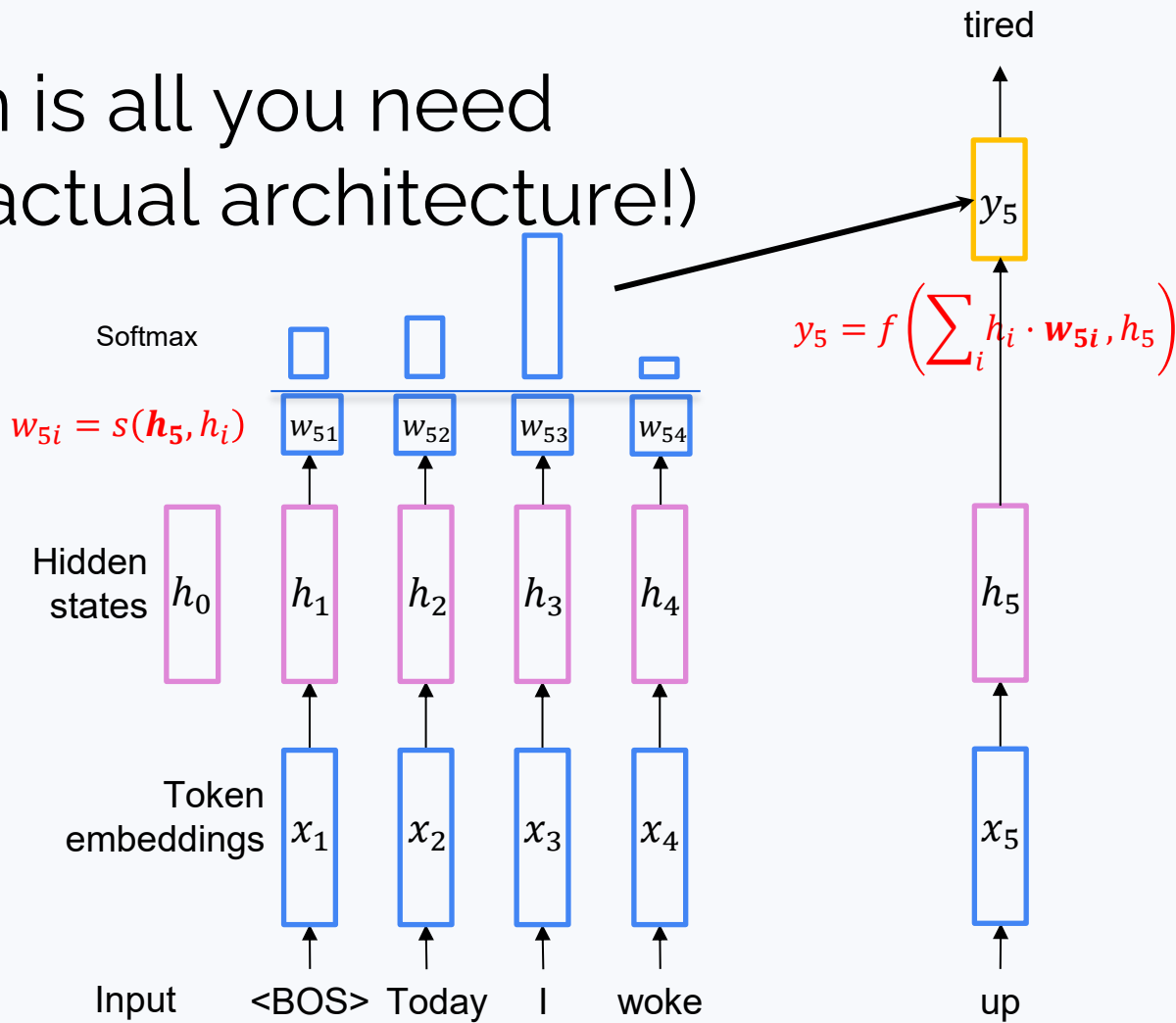
- $s(e, d) = e^T d$
- $s(e, d) = e^T W d$
- $s(e, d) = W_2 \tanh(W_1 \cdot [e, d])$



# Language modelling with attention

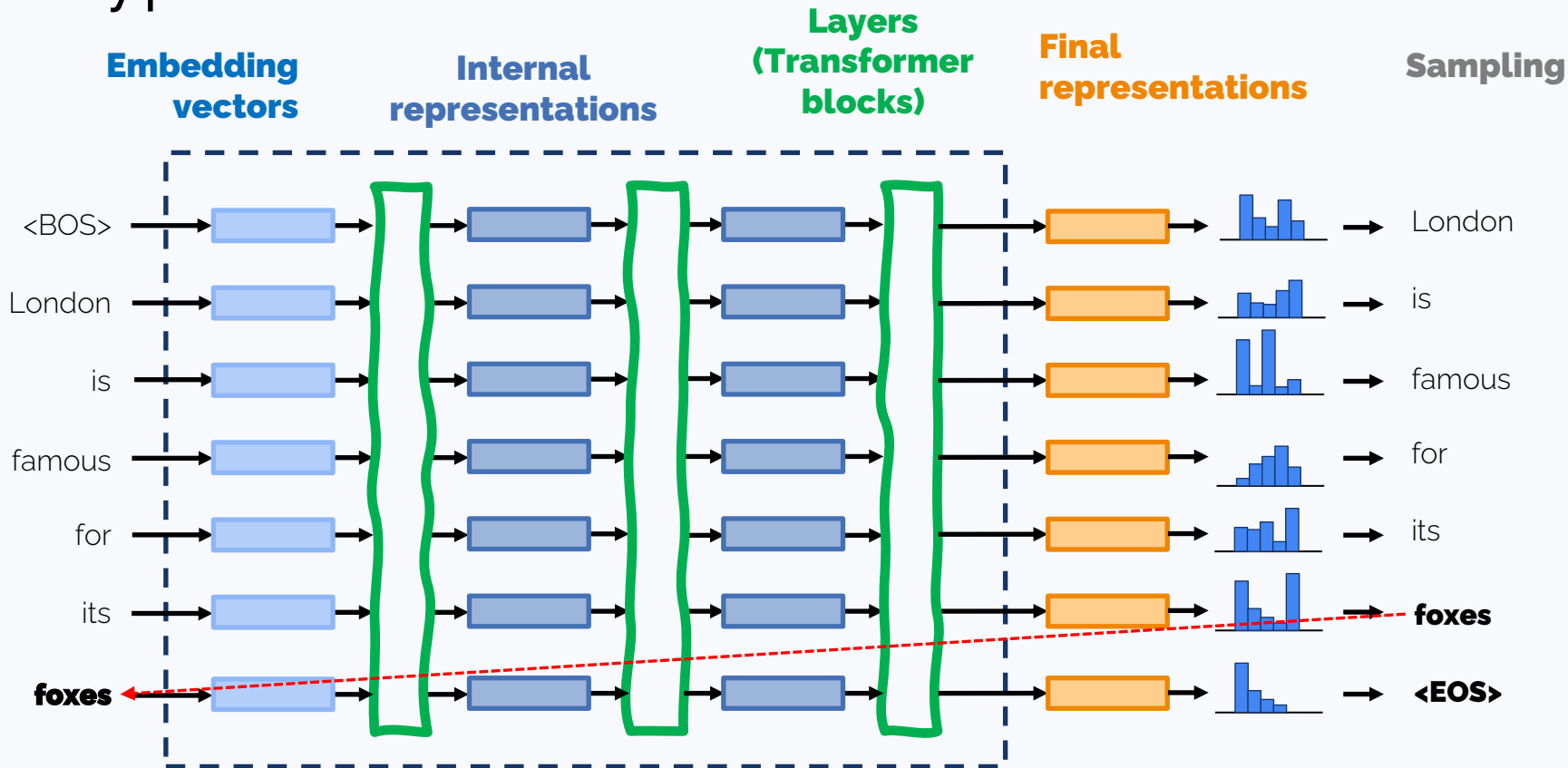


# Attention is all you need (not the actual architecture!)



# Transformer

# A typical LLM

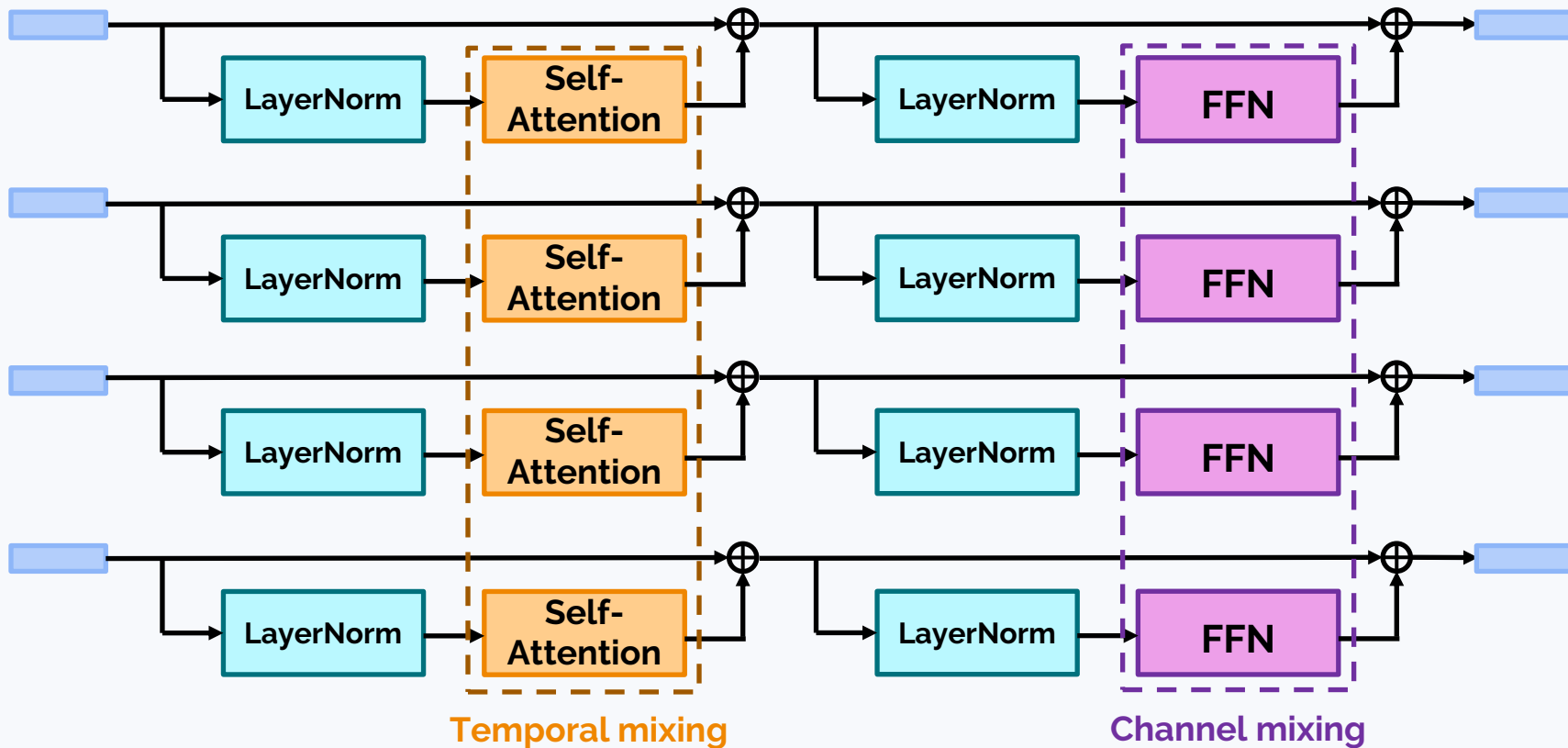


# Two roles of a transformer block

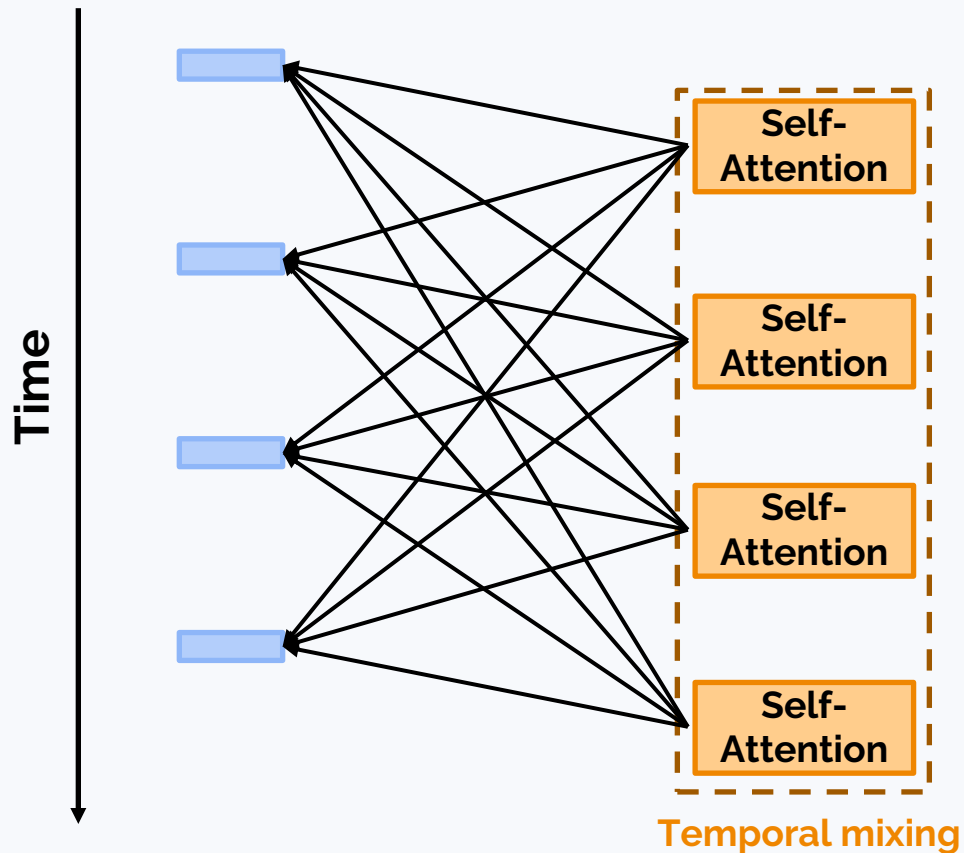
- **Temporal mixing** – information exchange between hidden states of different tokens across the sequence – **using context**
- **Channel mixing** – enriching of a hidden representation of each of the tokens – **applying knowledge**

(Almost) classic transformer block in an LLM:

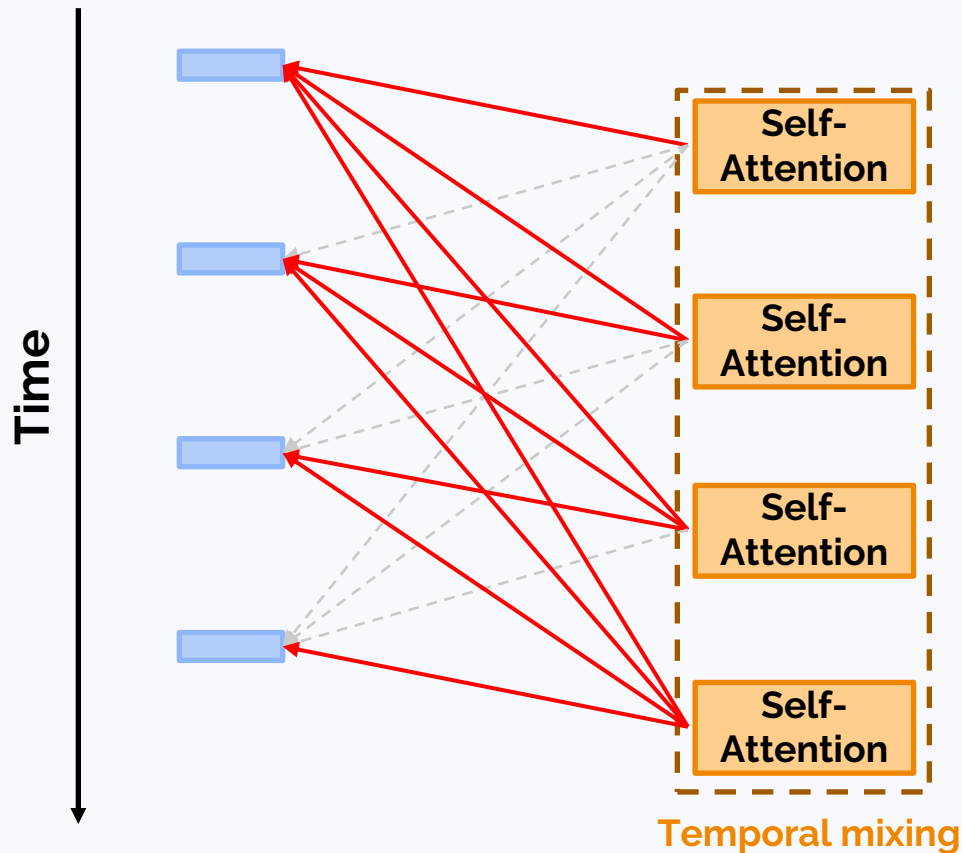
$$z = x + \text{SelfAttention}(\text{Norm}(x)) \quad \text{out} = z + \text{FFN}(\text{Norm}(z))$$



# The idea of self-attention

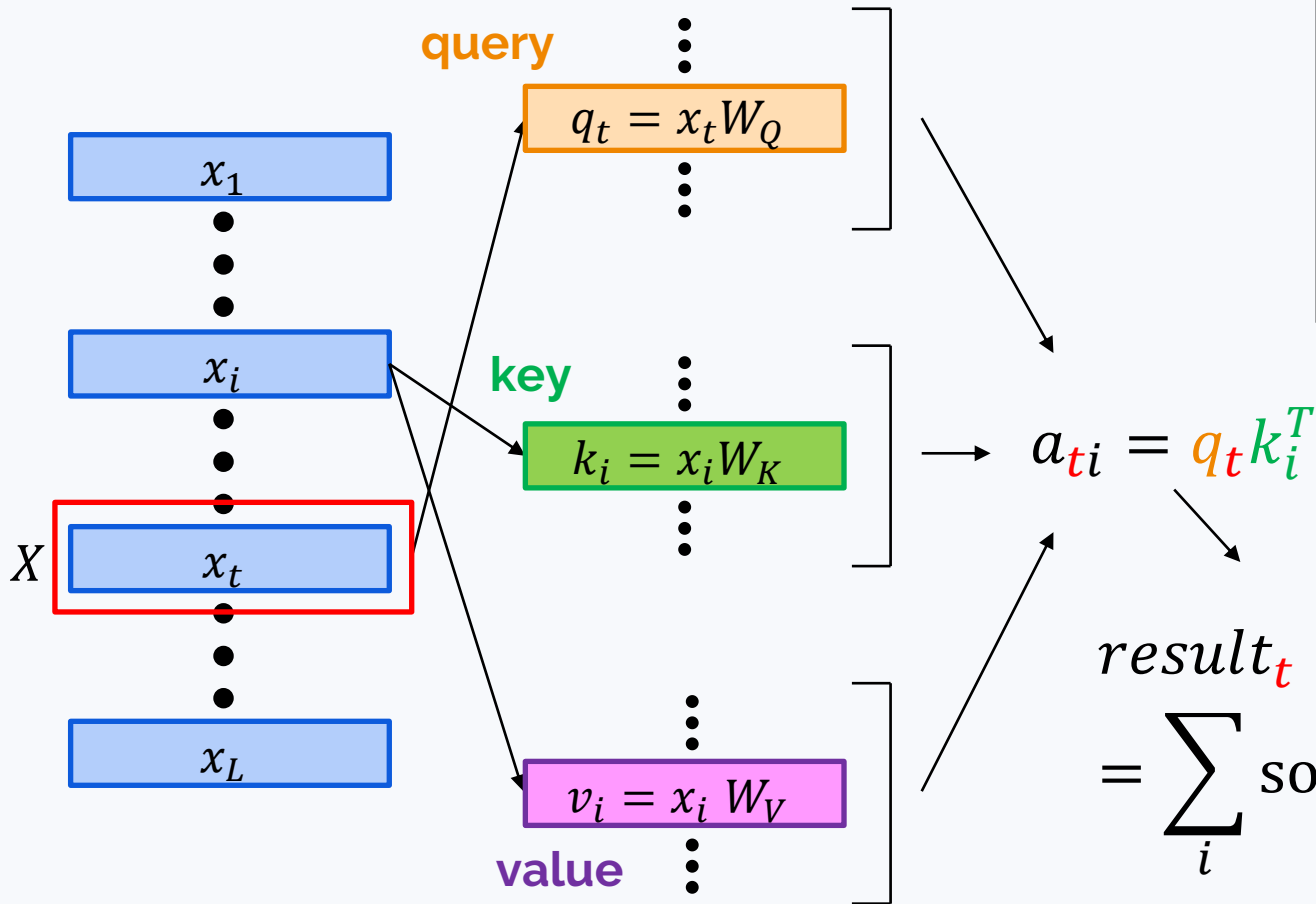


The idea of **masked** self-attention:  
don't look into the future





# QKV-attention



**query:** “I need this”

**key:** “I have this”

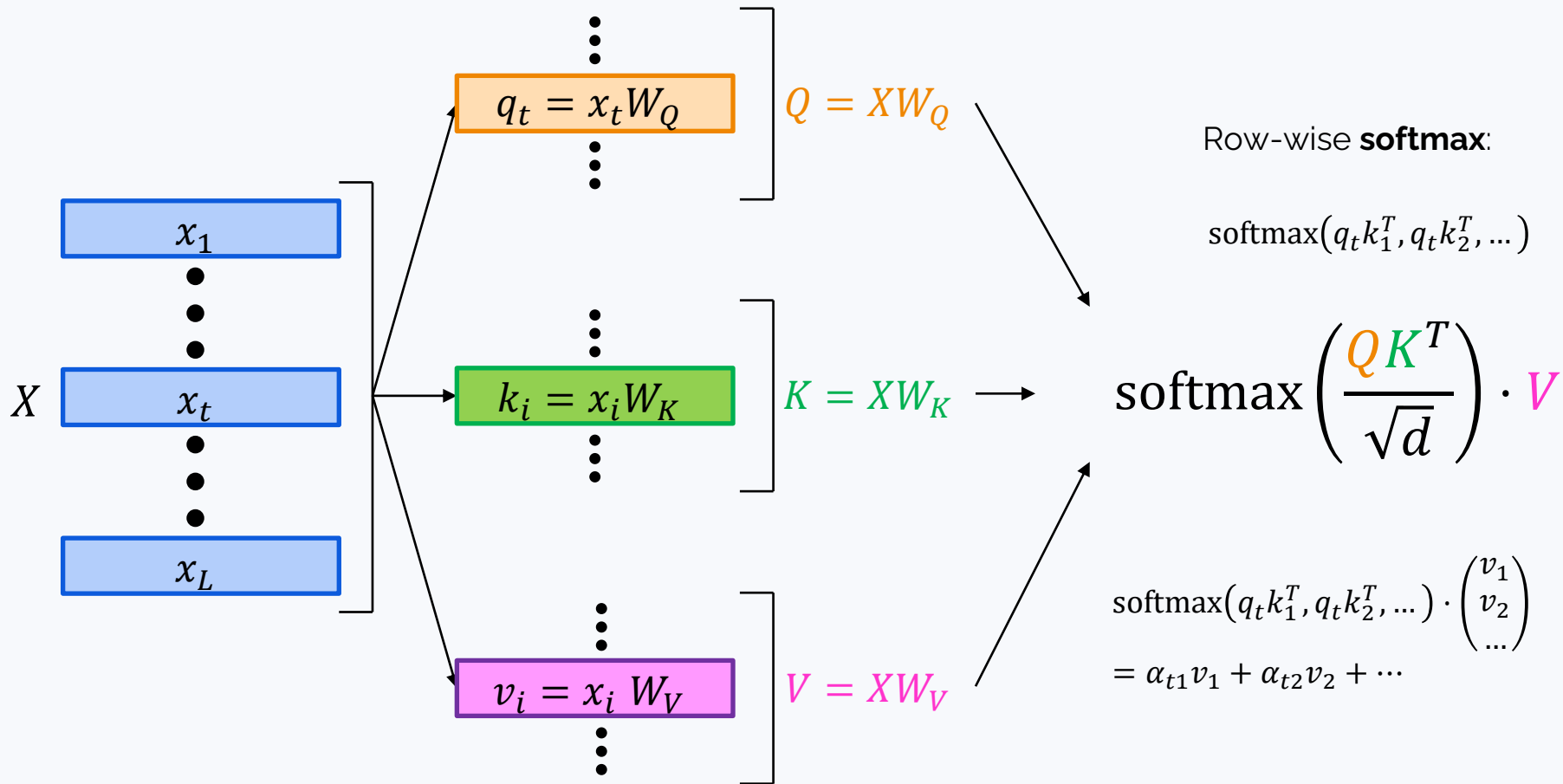
**attention score:** How much they align

**value:**

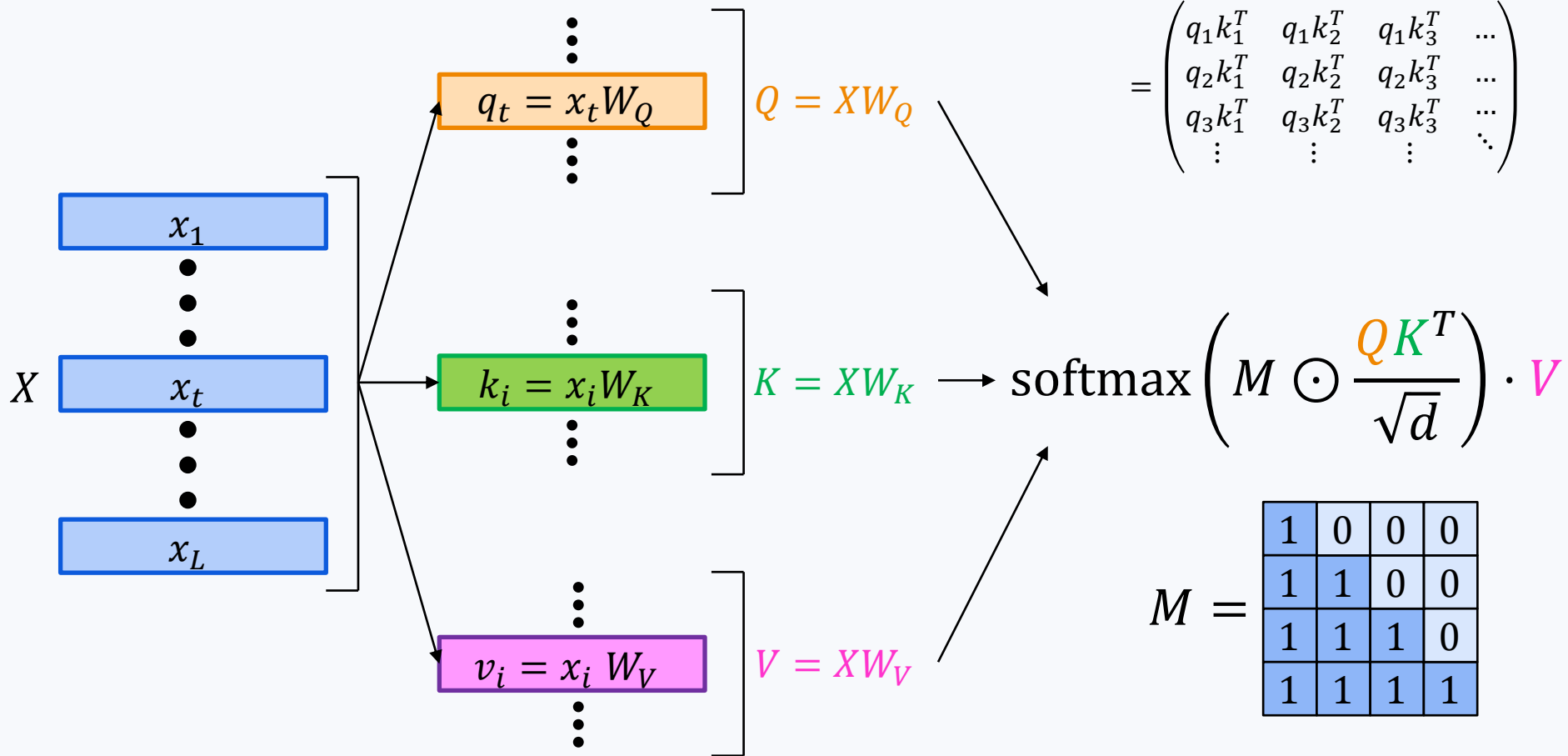
The information

$$result_t = \sum_i \text{softmax}(a_t)_i v_i$$

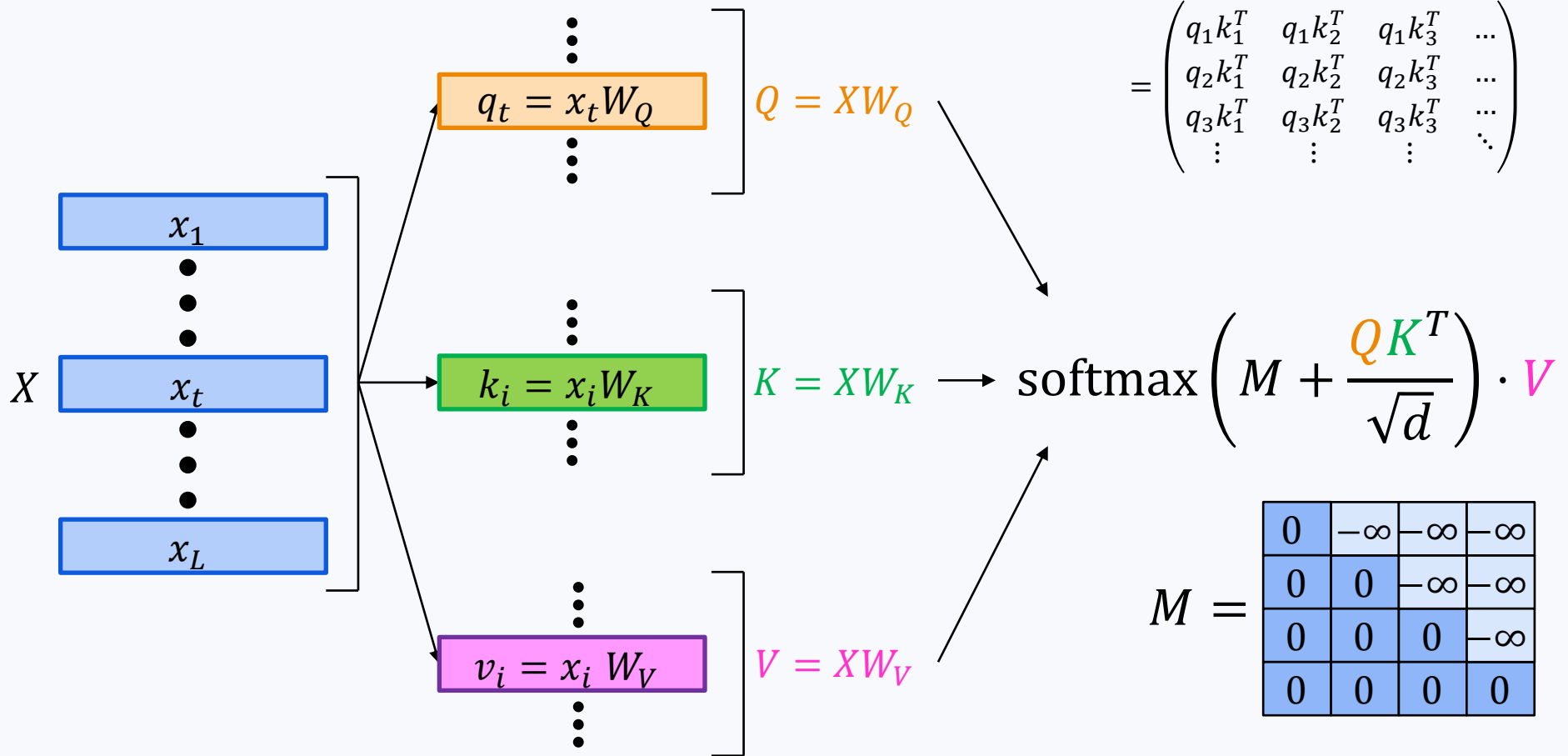
# QKV-attention



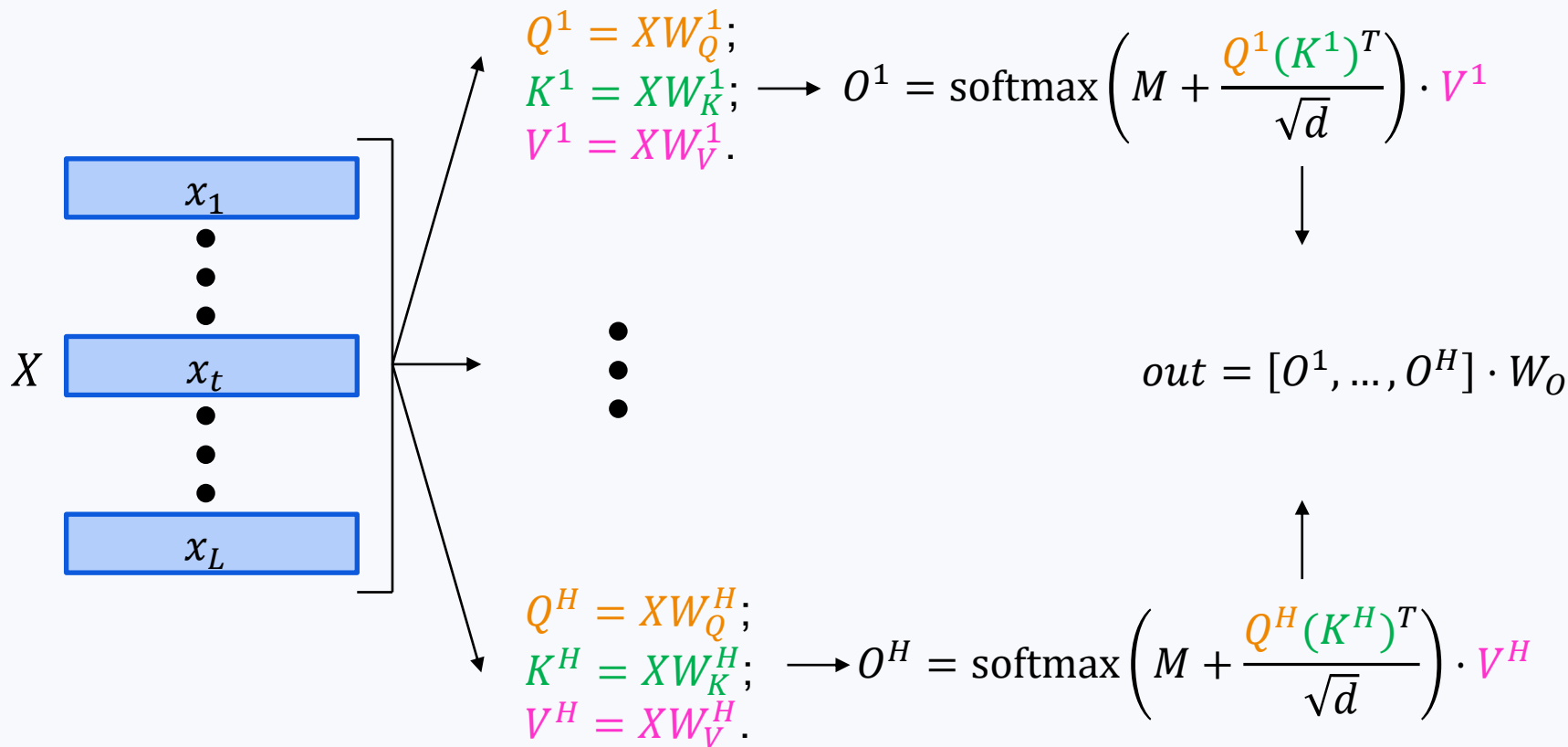
# Masked QKV-attention



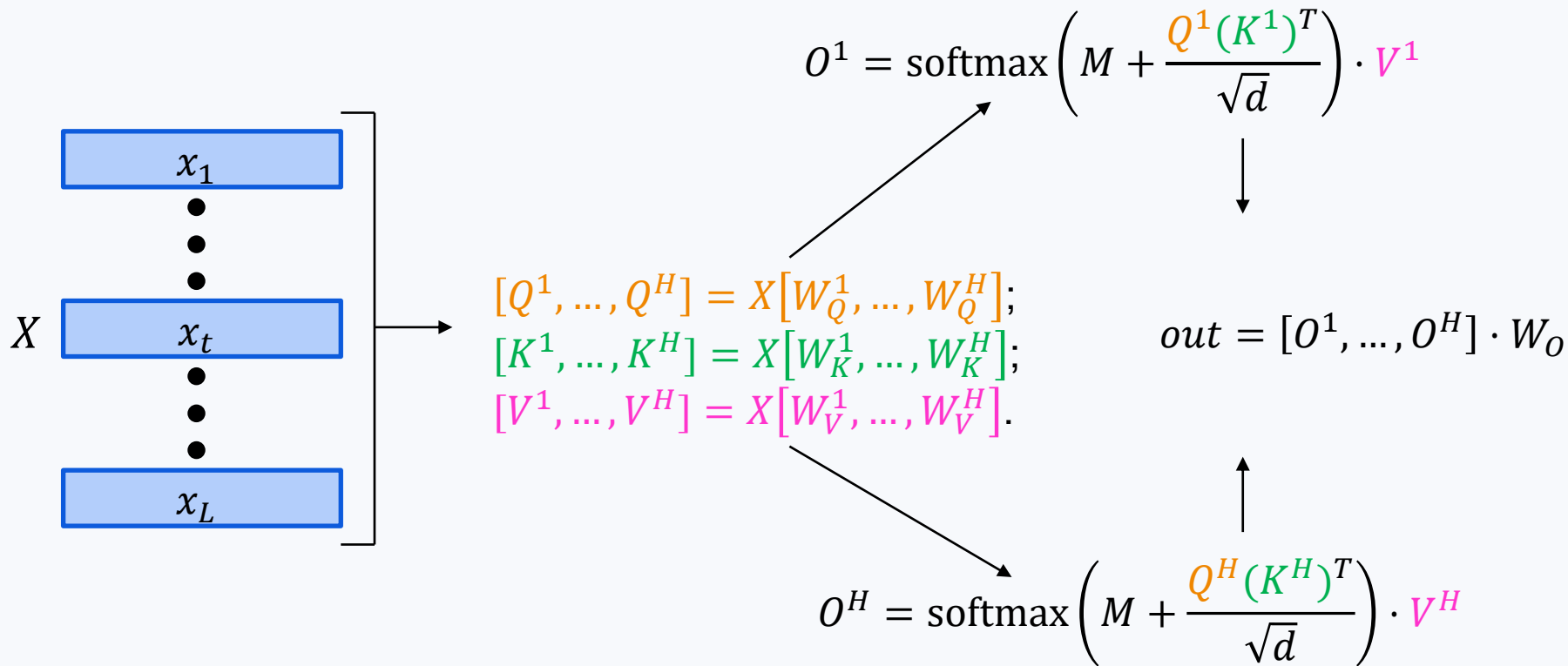
# Masked QKV-attention



# Multi-head attention

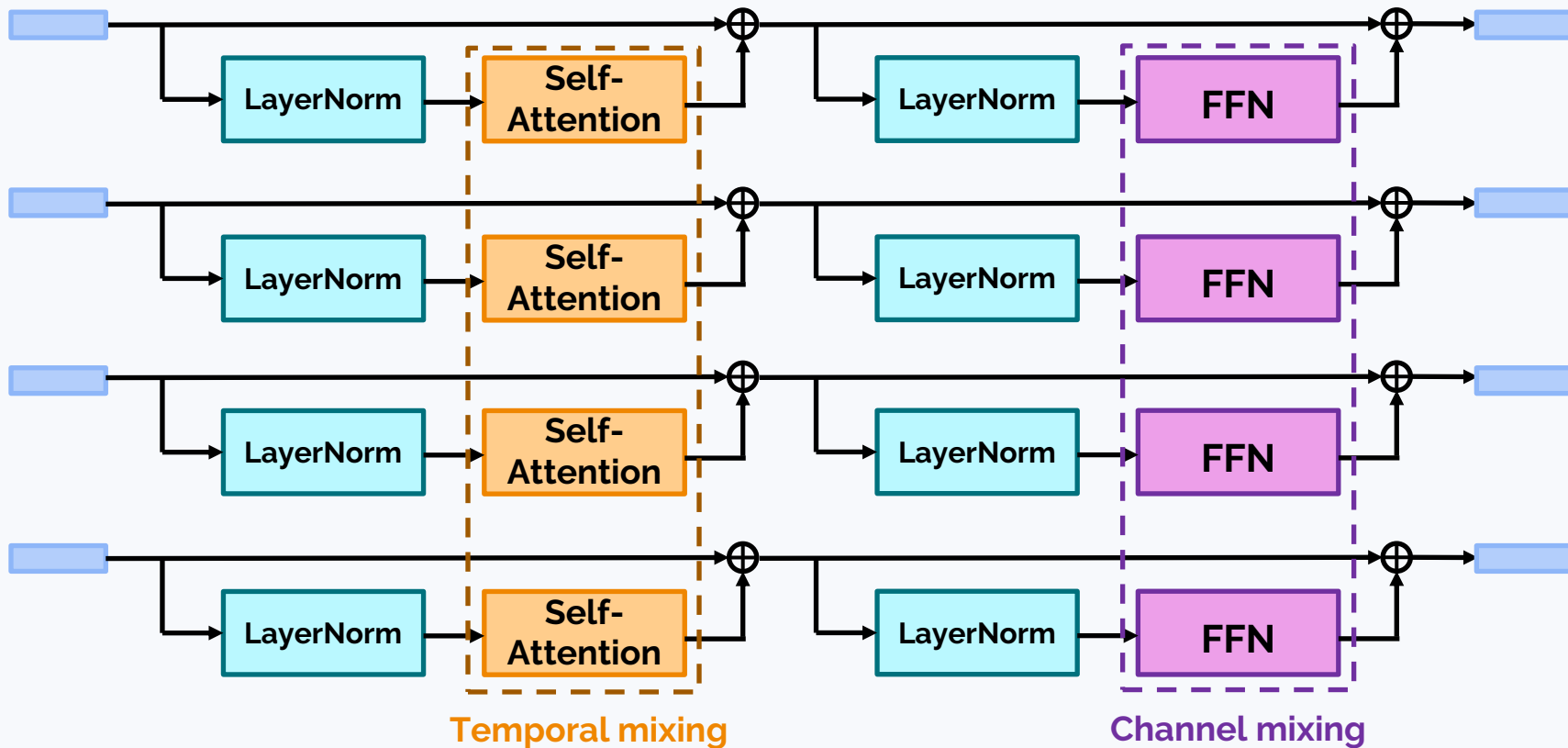


# Multi-head attention



(Almost) classic transformer block in an LLM:

$$z = x + \mathbf{SelfAttention}(\mathbf{Norm}(x)) \quad \text{out} = z + \mathbf{FFN}(\mathbf{Norm}(z))$$



# BatchNorm

During training:

$$x_{k+1} = \frac{x_k - \mu(\text{batch})}{\sqrt{\sigma^2(\text{batch}) + \varepsilon}},$$

Then

$$x_{k+2} = \beta x_{k+1} + \gamma,$$

where  $\beta$  and  $\gamma$  are trainable

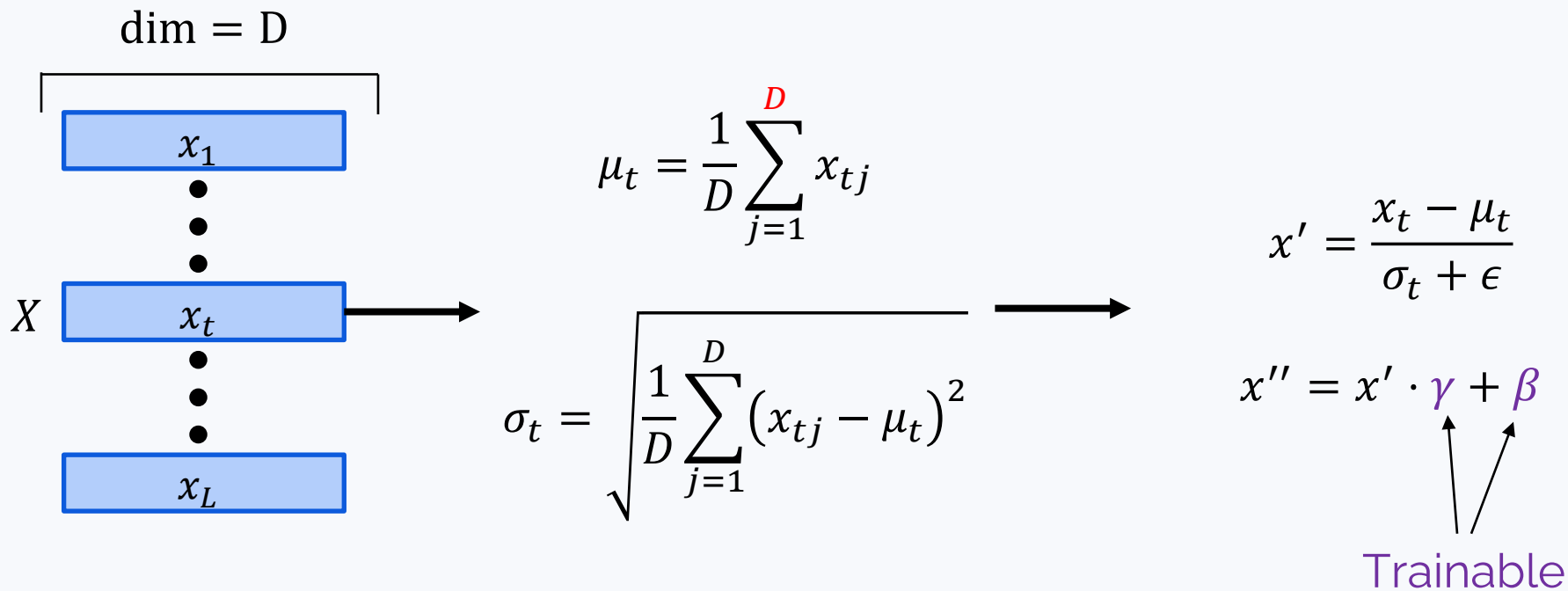
During inference:

$$\mu_* = \mu_* \lambda + \mu(\text{batch}) \cdot (1 - \lambda),$$

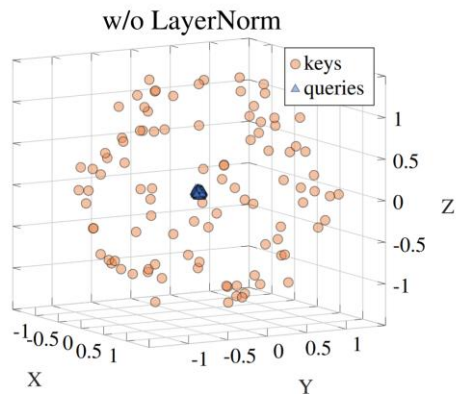
$$\sigma_*^2 = \sigma_*^2 \lambda + \sigma^2(\text{batch}) \cdot (1 - \lambda),$$



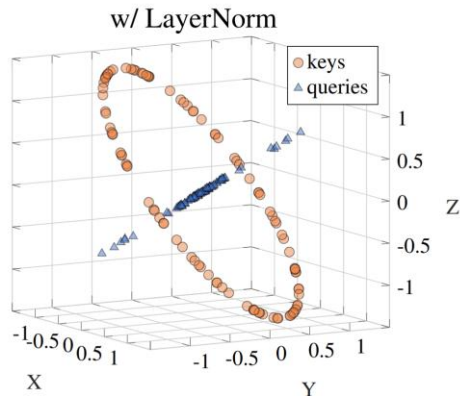
# LayerNorm



# Why (Pre-)LayerNorm works



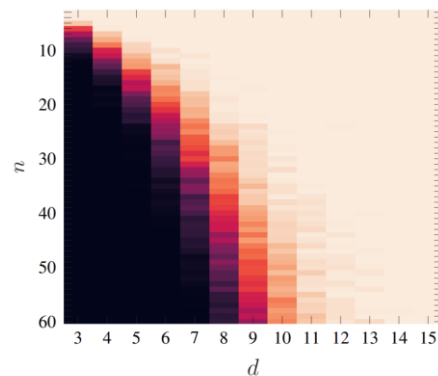
(a) Without LayerNorm, the model has learned key and query vectors without any apparent geometric structure.



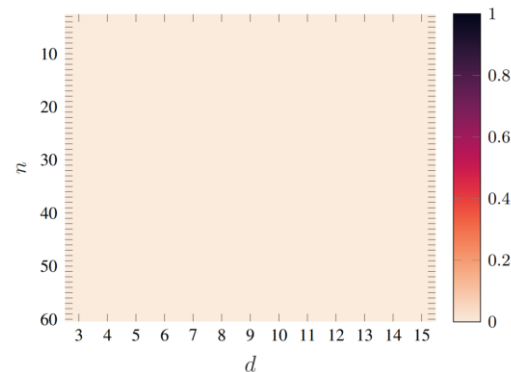
(b) LayerNorm projects the key vectors onto the same hyperplane so that the model can learn to align the queries to be orthogonal to the keys.

**Theorem 1.** Given a set of vectors  $\mathcal{S} = \{\mathbf{h}_1, \dots, \mathbf{h}_{n-1}, \mathbf{h}_n\}$  such that  $\mathbf{h}_n$  is interior to the convex hull of  $\mathcal{S}$ , then for all  $\mathbf{v} \in \mathbb{R}^d$  (s.t.  $\mathbf{v} \neq \vec{0}$ ):

$$\max_{i \in [n-1]} \mathbf{v}^\top \mathbf{h}_i > \mathbf{v}^\top \mathbf{h}_n$$



(c) Without LayerNorm, there are “unselectable” key vectors that cannot be selected by getting the maximal attention score (marked in darker colors).

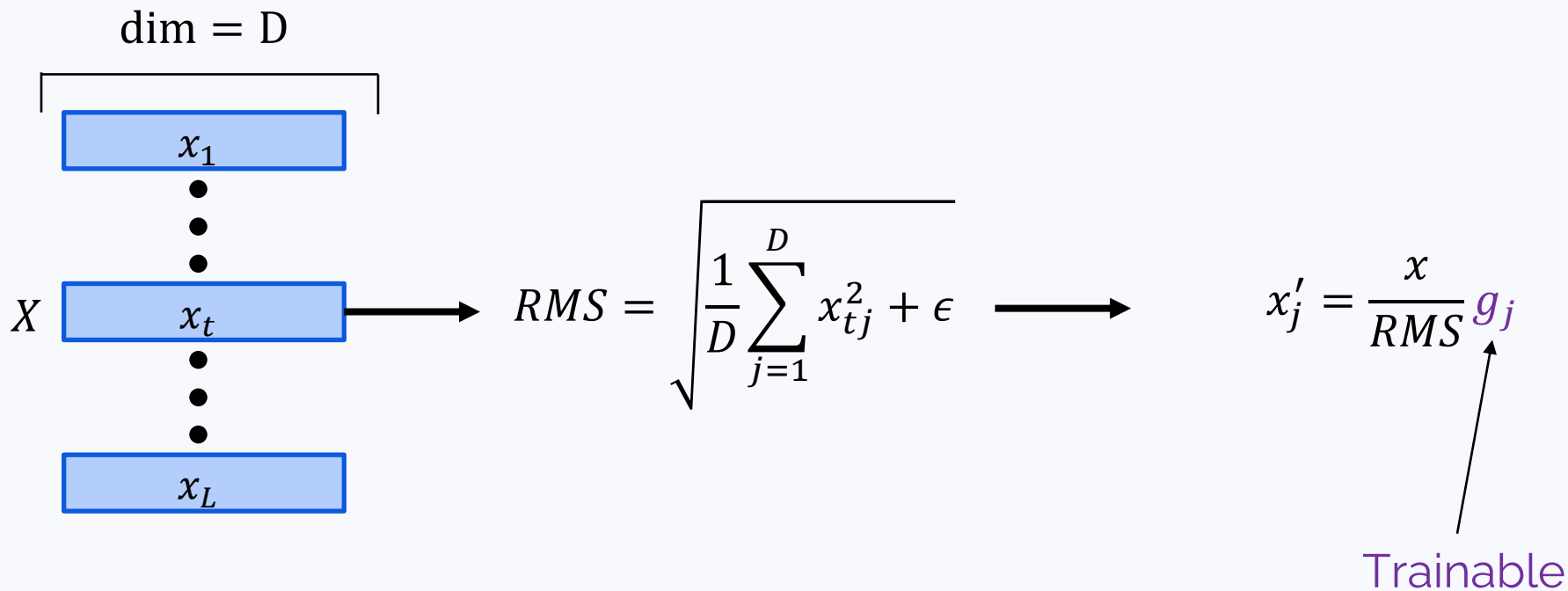


(d) LayerNorm eliminates the problem of “unselectable” key vectors: Applying LayerNorm allows any key to get the highest attention score.

<https://lessw.medium.com/what-layernorm-really-does-for-attention-in-transformers-4901ea6d890e>

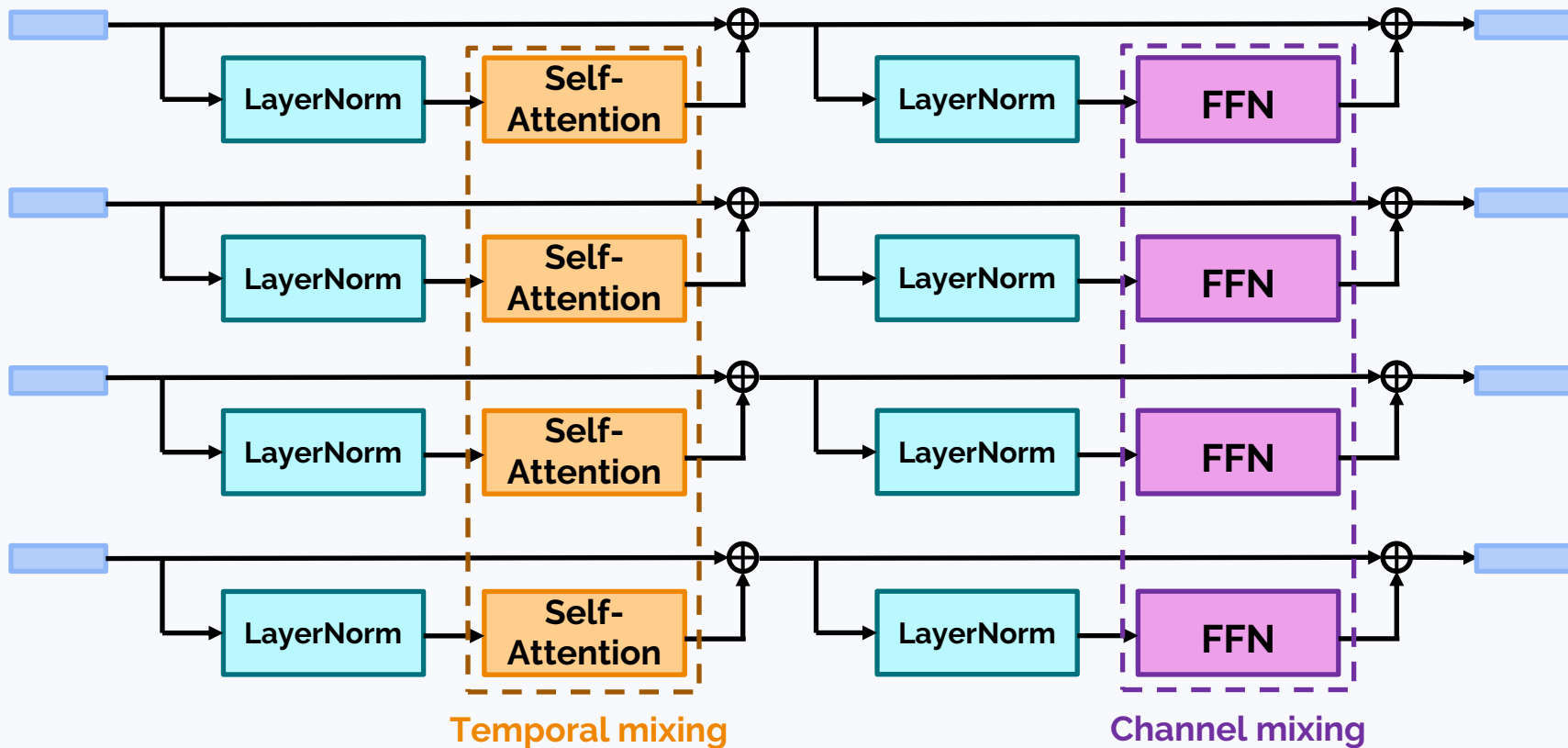
On the Expressivity Role of LayerNorm in Transformers' Attention <https://arxiv.org/abs/2305.02582>

# RMSNorm



(Almost) classic transformer block in an LLM:

$$z = x + \text{SelfAttention}(\text{Norm}(x)) \quad \text{out} = z + \text{FFN}(\text{Norm}(z))$$



Attention!

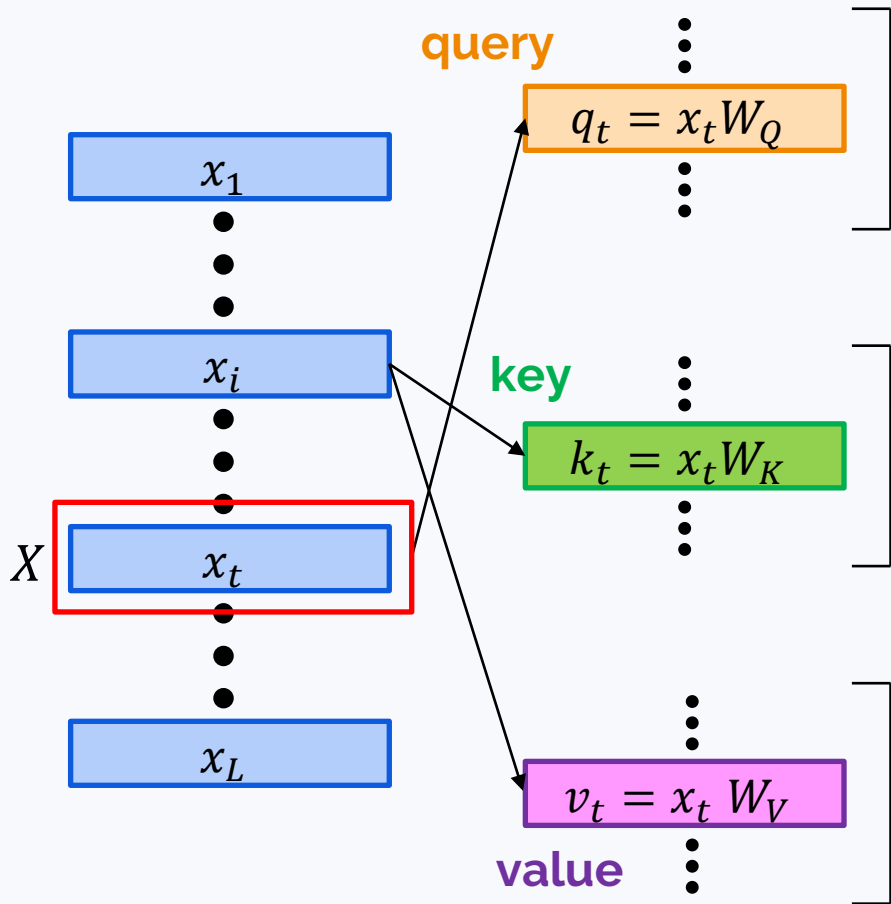
# Two stages of LLM inference

An LLM gets a **prompt**. Then:

- **Pre-fill**: prompt processing; we have it all, so we can just compute  $\text{softmax}\left(M + \frac{QK^T}{\sqrt{d}}\right) \cdot V$ . It has quadratic complexity due to matrix multiplication (even worse, memory-wise).
- **Autoregressive generation** – we generate new tokens one by one, each time computing  $\text{softmax}\left(M + \frac{q_t K^T}{\sqrt{d}}\right) \cdot V$

See any trouble here?

# QKV-attention



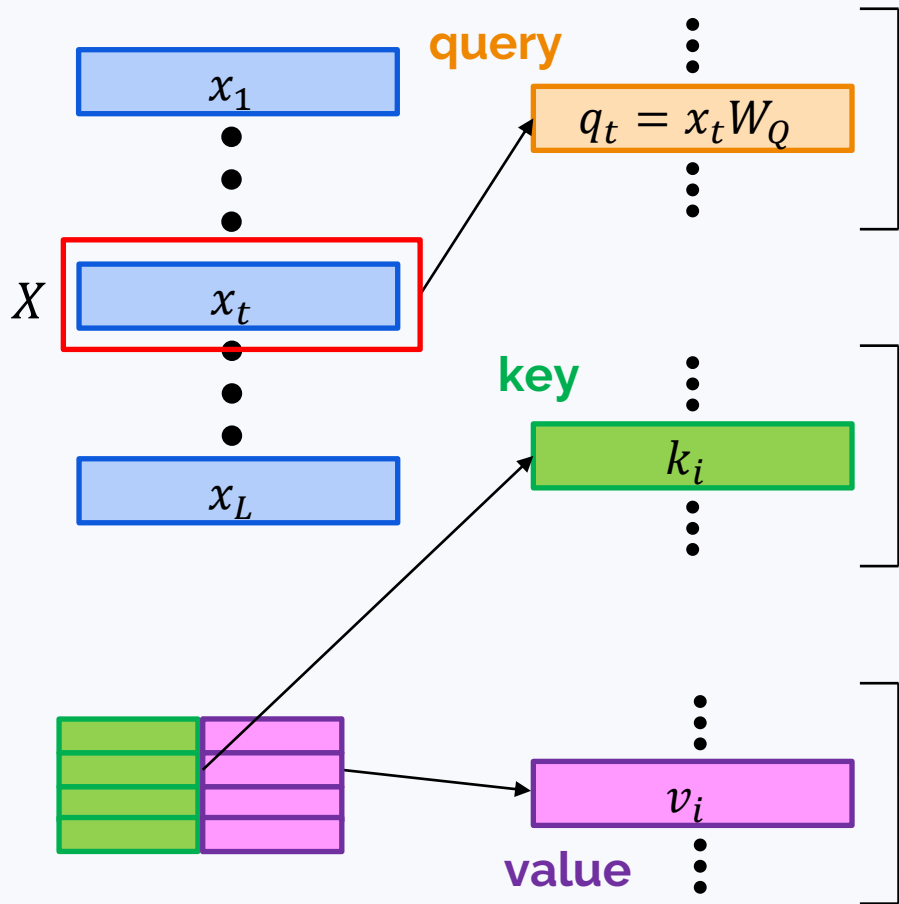
**query**: comes from  $x_t$   
**keys** and **values**: from  $1 \dots t$

**We have to recompute  
keys and values every  
single time!**

$$a_{ti} = q_t k_i^T$$

$$result_t = \sum_i a_{ti} v_i$$

# KV cache



No recomputations!

The price:

**Memory requirements grow linearly with the sequence length**

$$a_{ti} = q_t k_i^T$$

$$result_t = \sum_i a_{ti} v_i$$

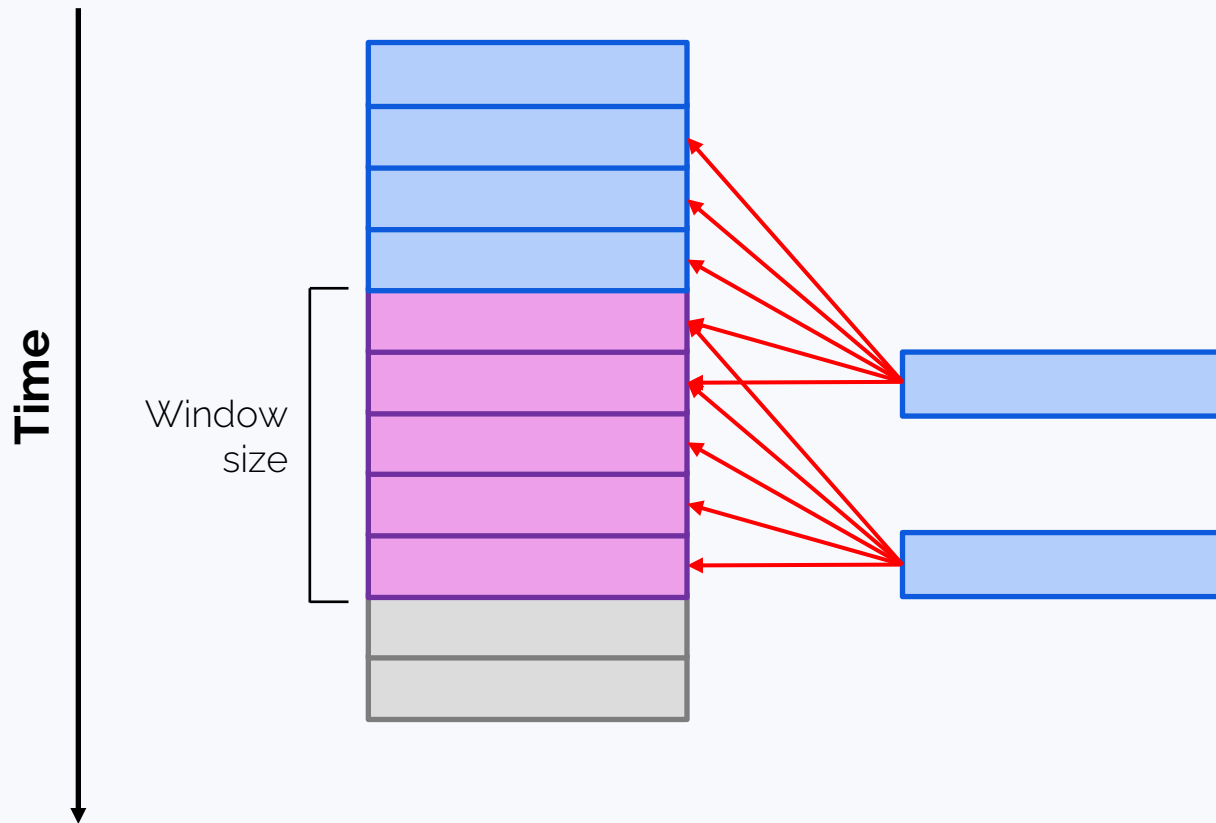


# Two stages of LLM inference

An LLM gets a **prompt**. Then:

- **Cache pre-fill**: prompt processing; we have it all, so we can just compute  $\text{softmax}\left(M + \frac{QK^T}{\sqrt{d}}\right) \cdot V$ . We fill KV-cache for the whole **prompt**. It has quadratic complexity due to matrix multiplication (even worse, memory-wise).
- **Autoregressive generation** – we generate new tokens one by one, each time computing  $\text{softmax}\left(M + \frac{q_t K^T}{\sqrt{d}}\right) \cdot V$ . We keep updating the KV-cache.

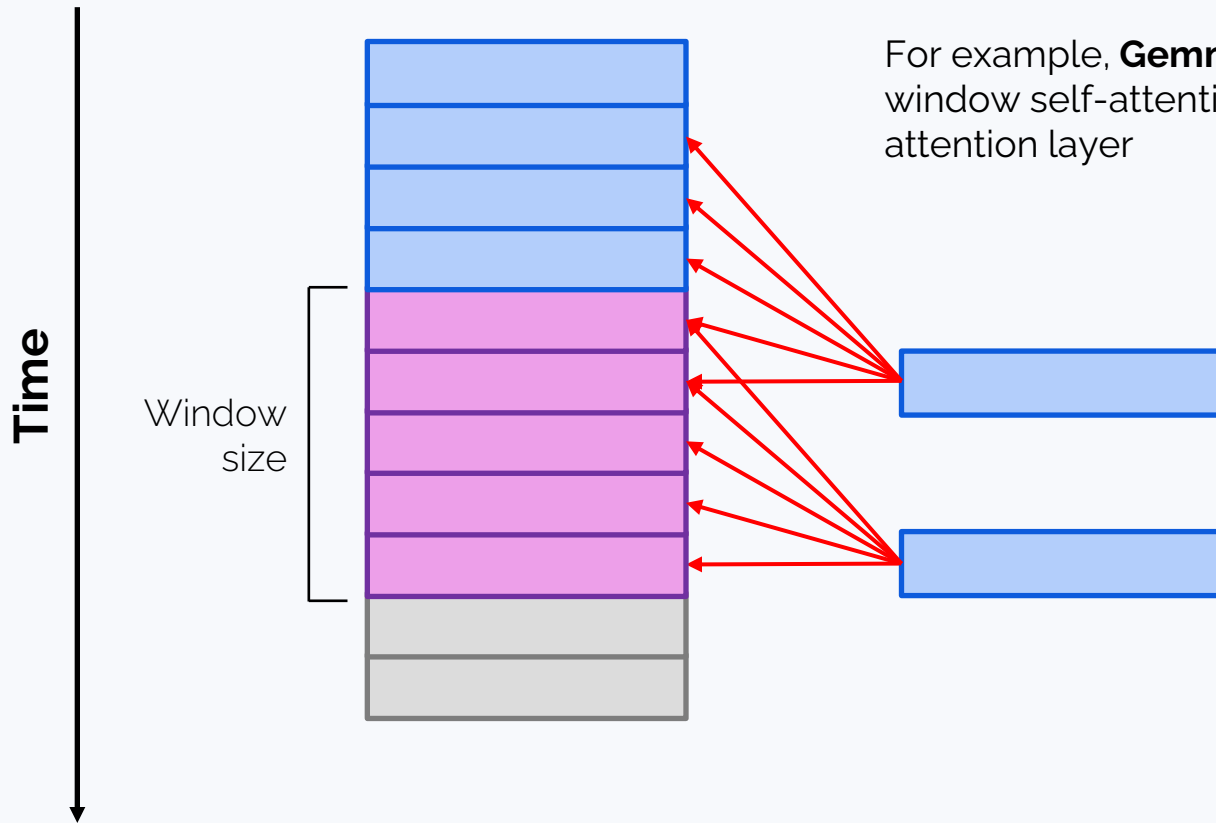
# Sliding window attention



# Sliding window attention

## Hybrid models

For example, **Gemma 3** has alternating 5 local sliding window self-attention layers for every global self-attention layer



# Group query attention

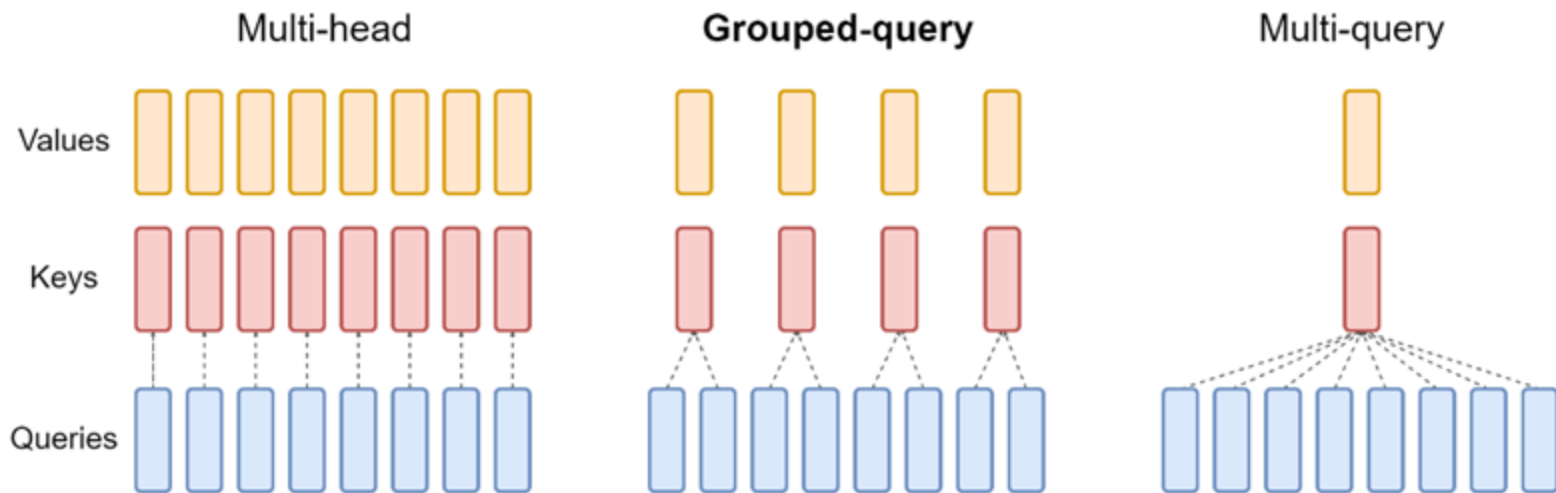


Figure 2: Overview of grouped-query method. Multi-head attention has  $H$  query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

# Flash attention

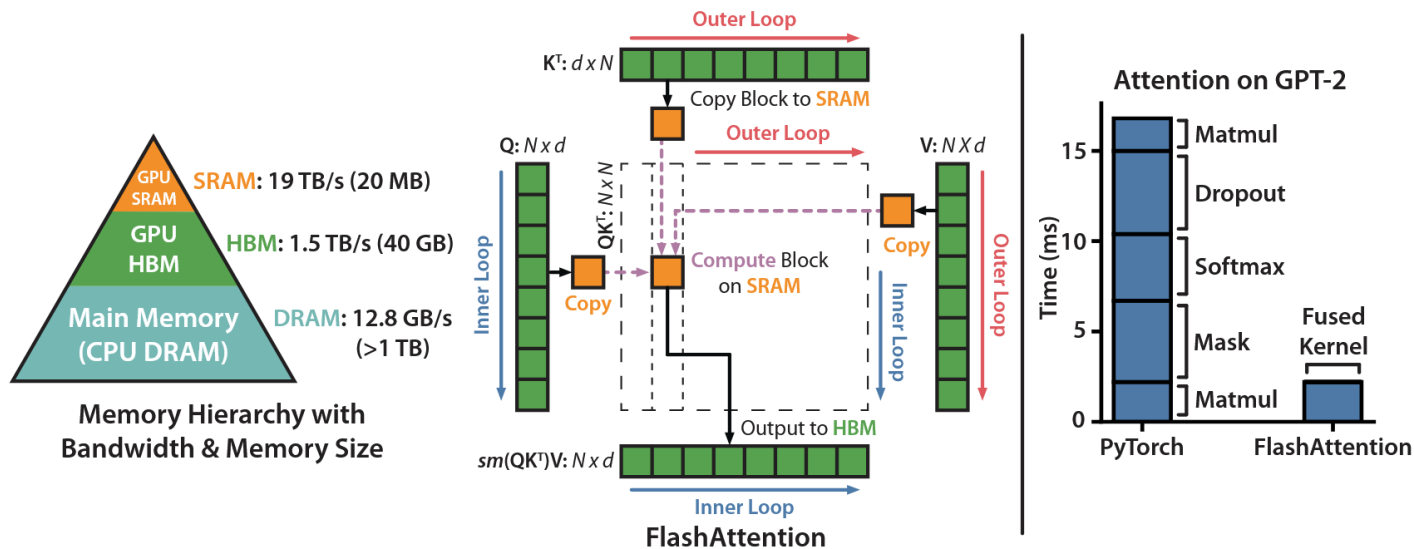
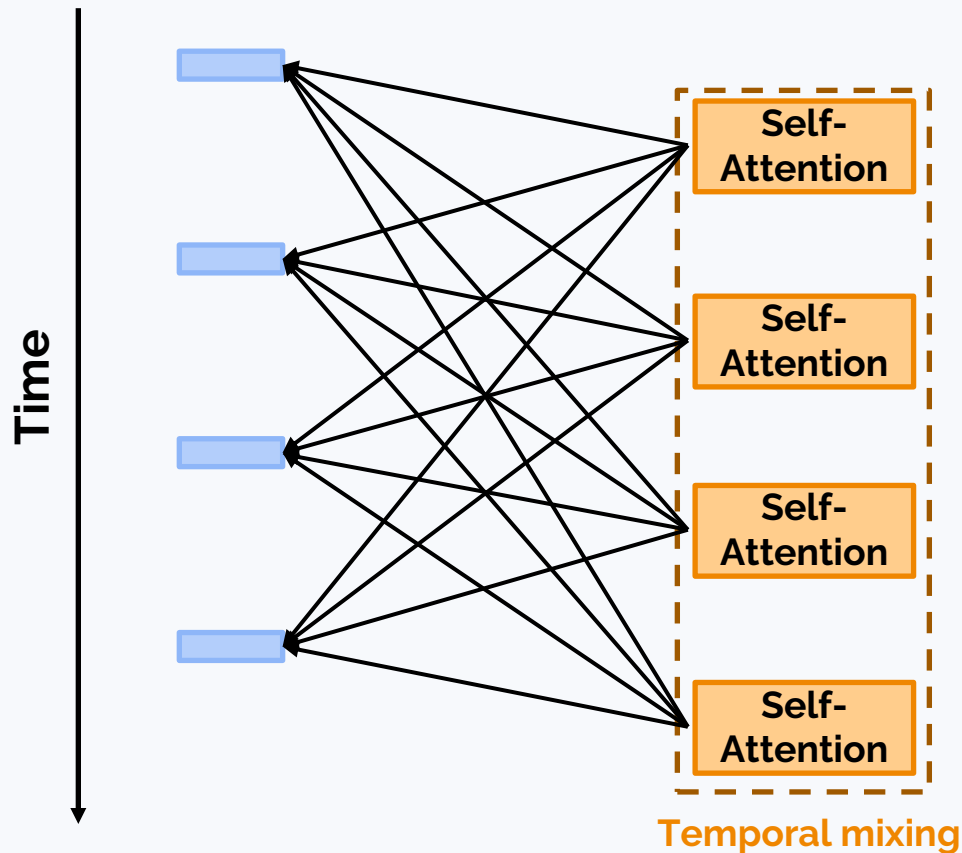


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large  $N \times N$  attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the  $K$  and  $V$  matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of  $Q$  matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large  $N \times N$  attention matrix to HBM, resulting in an 7.6 $\times$  speedup on the attention computation.

Position encoding

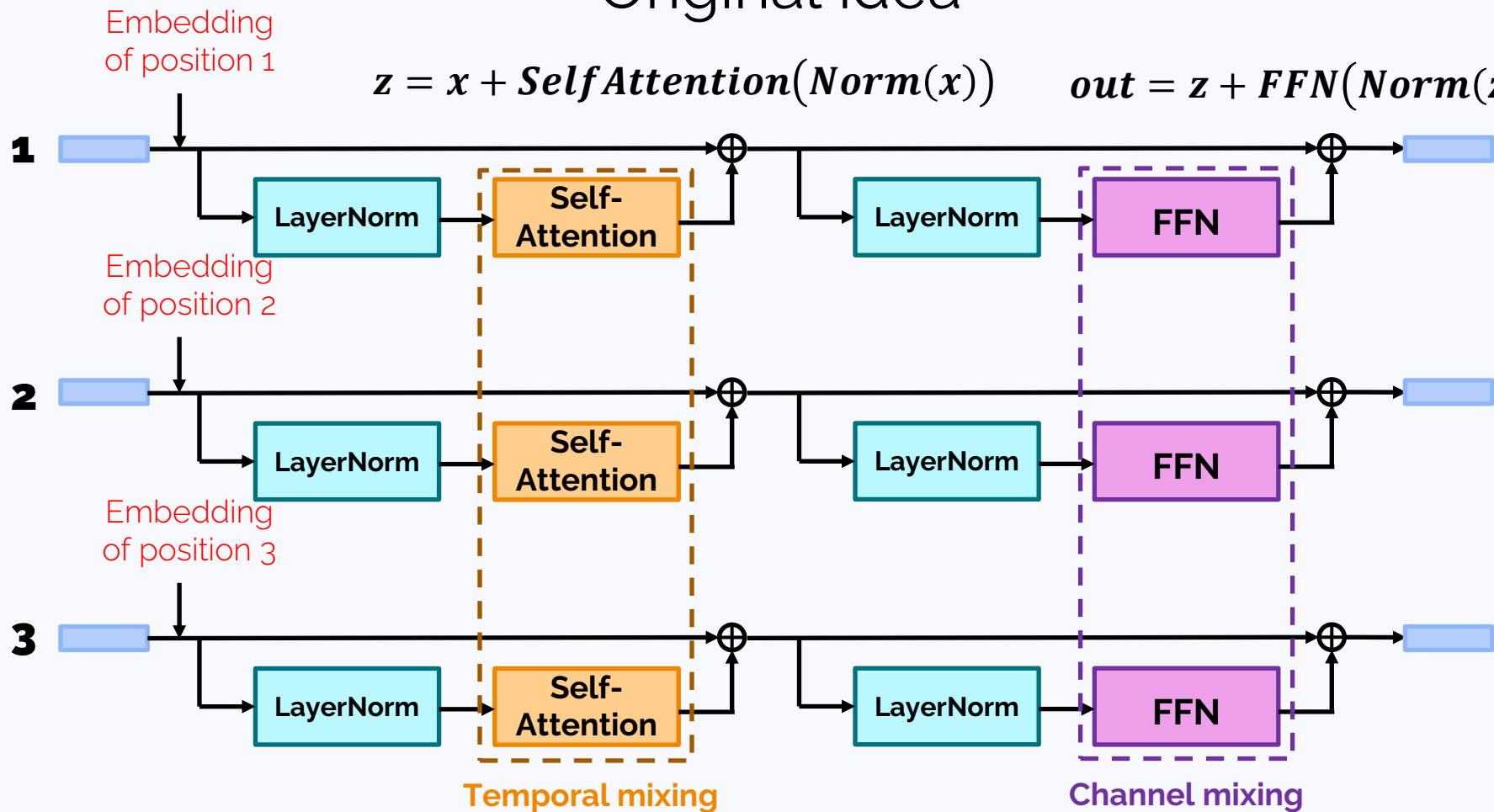
# Self-attention is totally oblivious to token order!



# Original idea

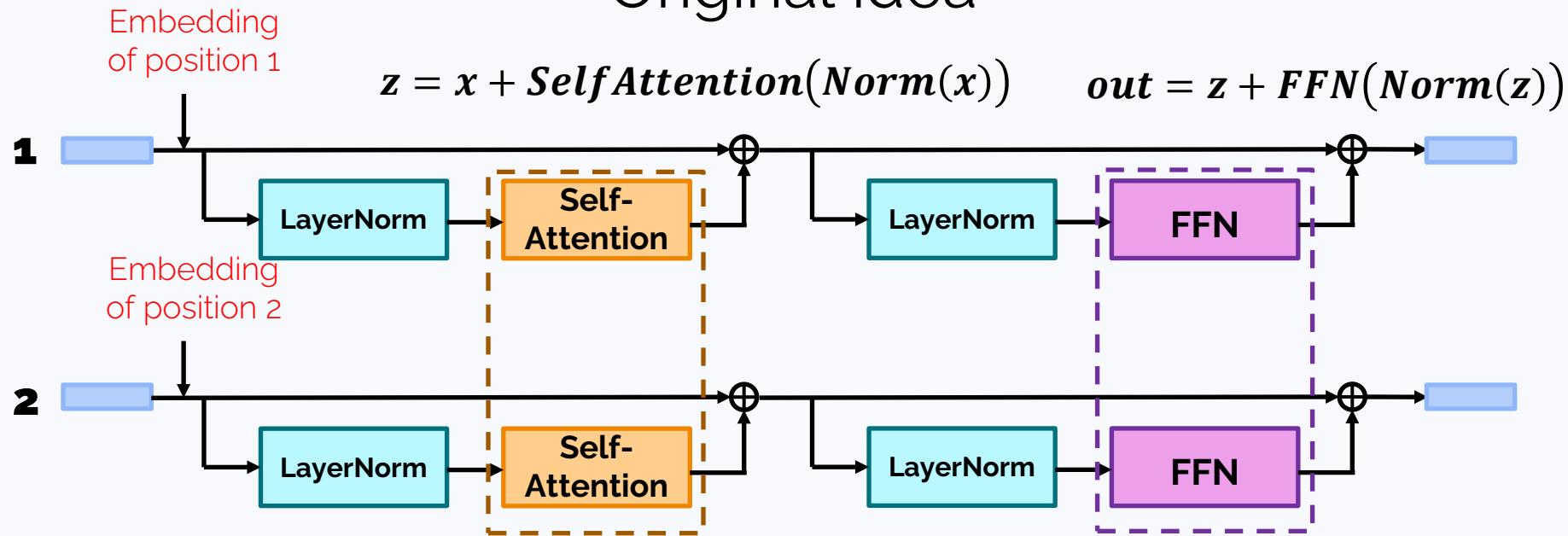
$$z = x + \text{SelfAttention}(\text{Norm}(x))$$

$$\text{out} = z + \text{FFN}(\text{Norm}(z))$$





# Original idea



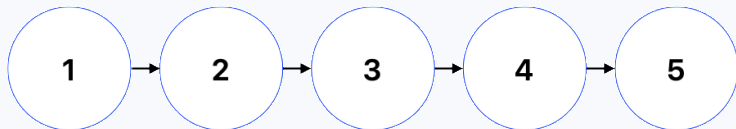
$$PE(t, 2i) = \sin\left(\frac{t}{10000^{i/d}}\right)$$

$$PE(t, 2i + 1) = \cos\left(\frac{t}{10000^{i/d}}\right)$$

## Positional encoding

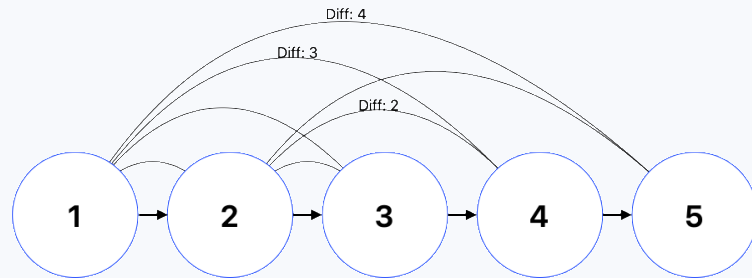
### Absolute

Encodes absolute positions of tokens using fixed embeddings added to the input.



### Relative

Encodes positional relationships between tokens, often applied in self-attention



# Rotary positional encoding (RoPE): the idea

Let  $q_t, k_m$  be two-dimensional. When we compute  $q_t k_m^T$ , do instead:

$$(q_t e^{it}) \cdot (k_m e^{im})^T = q_t e^{it} e^{-im} k_m^T = q_t e^{i(\textcolor{red}{t}-\textcolor{red}{m})} k_m^T$$

Where we denote

$$e^{in} = \begin{pmatrix} \cos n\theta & \sin n\theta \\ -\sin n\theta & \cos n\theta \end{pmatrix}$$

# Rotary position embedding

We use a block-diagonal rotation matrix:

$$f_q(x_m, m) = x_m W_Q R_{\Theta, m}^d, \quad f_k(x_n, n) = x_n W_K R_{\Theta, n}^d,$$

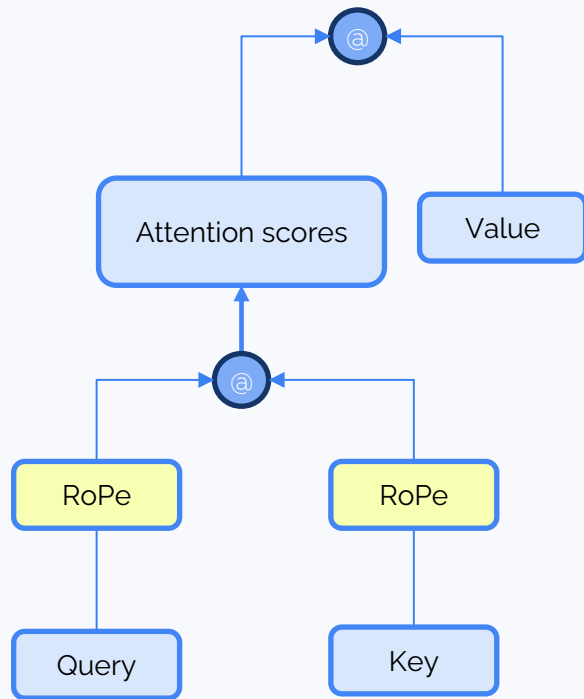
where

$$R_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & \sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ -\sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & \sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & -\sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & \sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & -\sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix},$$

where the parameters  $\Theta$  are set to

$$\theta_i = \theta_{\text{base}}^{-2(i-1)/d}.$$

Usually,  $\theta_{\text{base}}$  is large, such as 10000



# Long-term decay

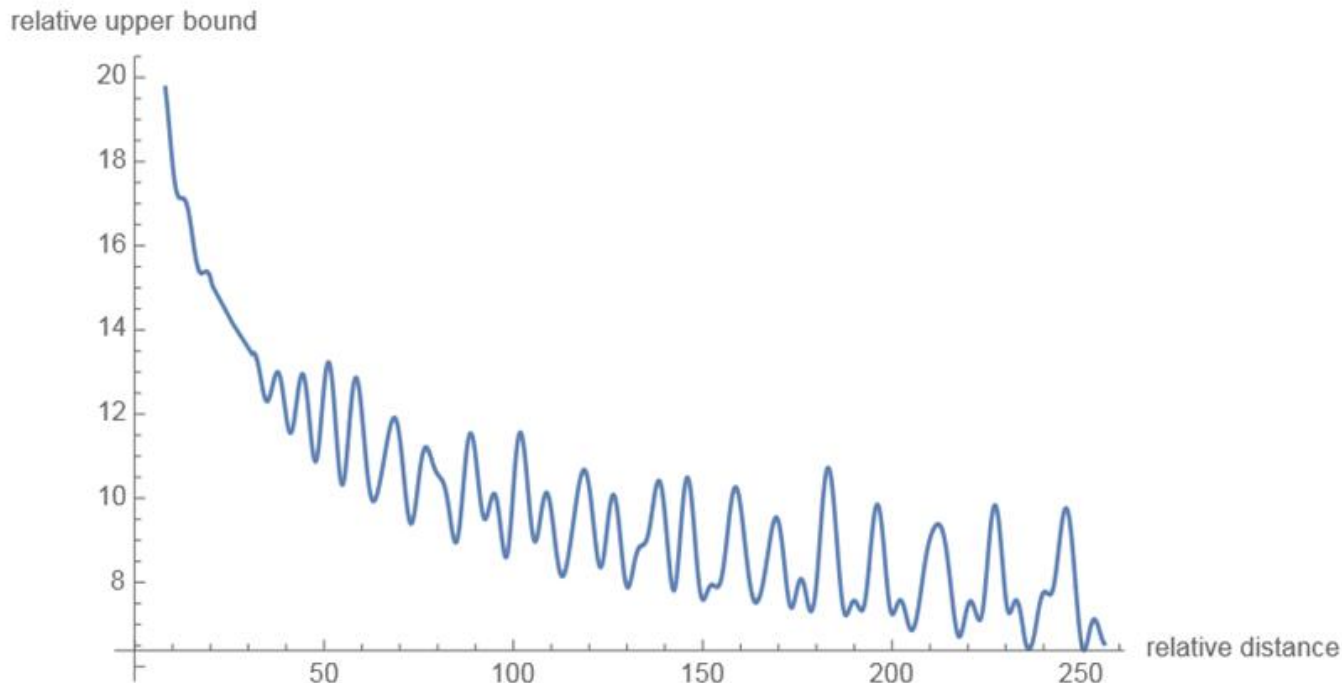
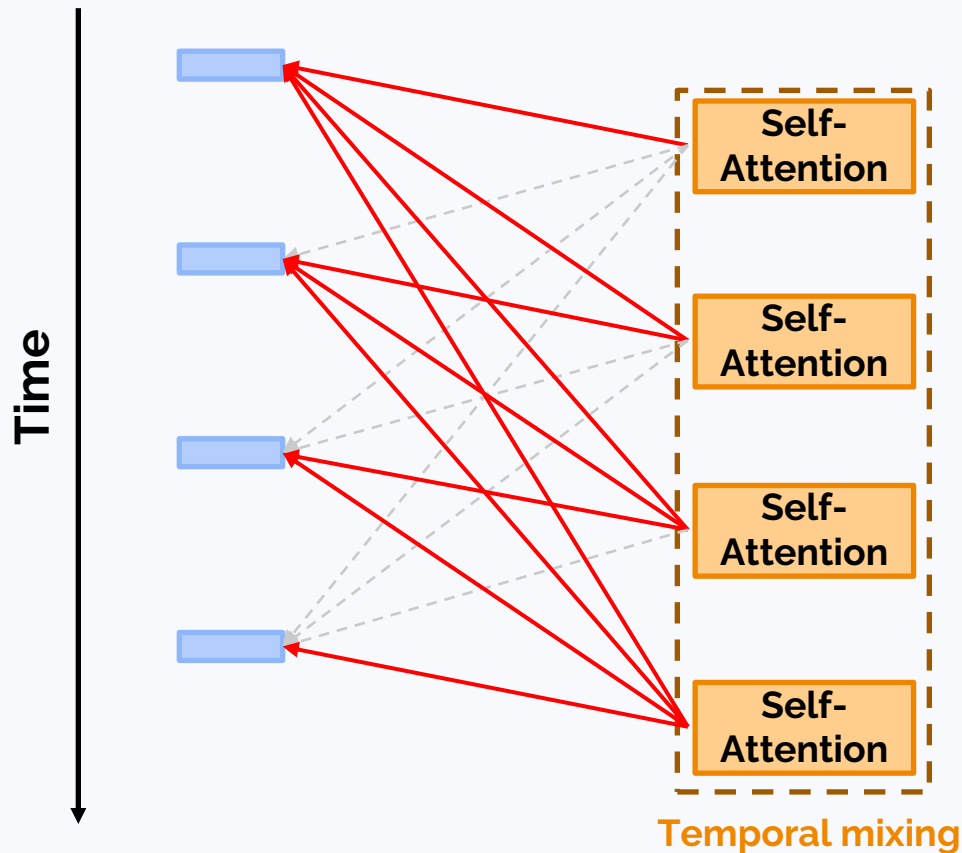


Figure 2: Long-term decay of RoPE.

# Masked self-attention isn't oblivious to order

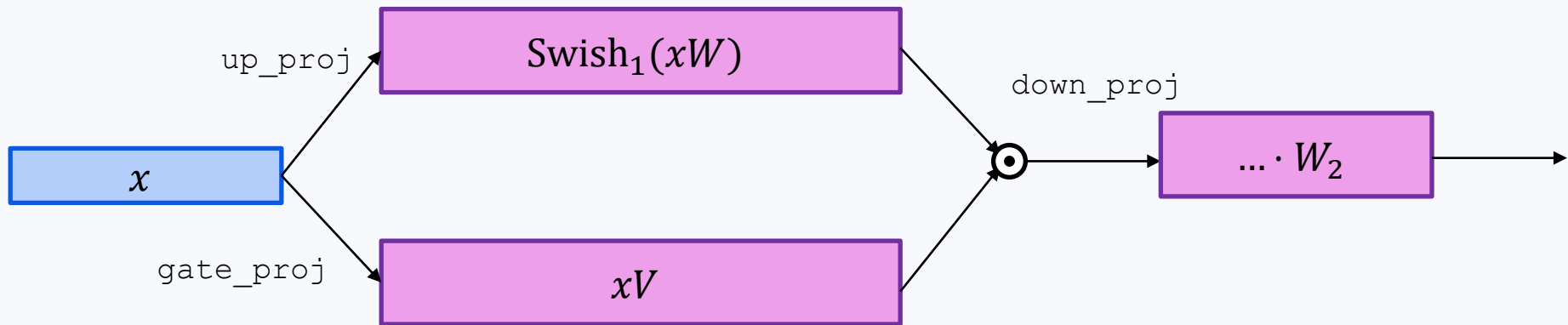


FFN

# Gated MLP (S<sub>wi</sub>GLU)

$$FFN(x, W, V, W_2) = (\text{Swish}_1(xW) \odot xV)W_2$$

$$\text{Swish}_\beta(x) = x \cdot \sigma(\beta x)$$



Usually,  $\dim(xW) \gg \dim x$



# Mixture of Experts (MoE)

<https://huggingface.co/blog/moe>

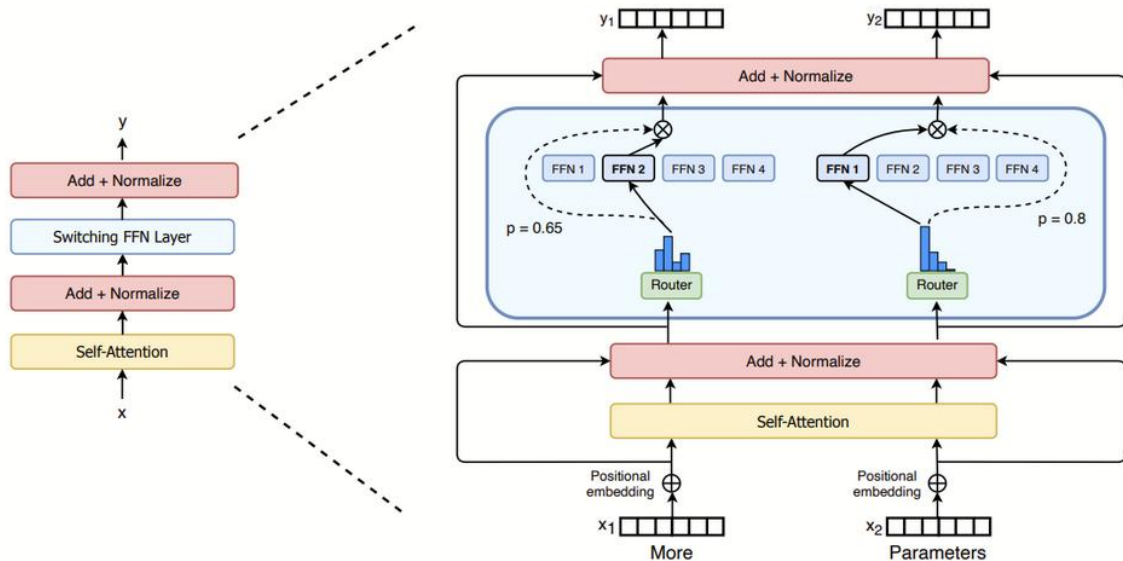


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens ( $x_1$  = "More" and  $x_2$  = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

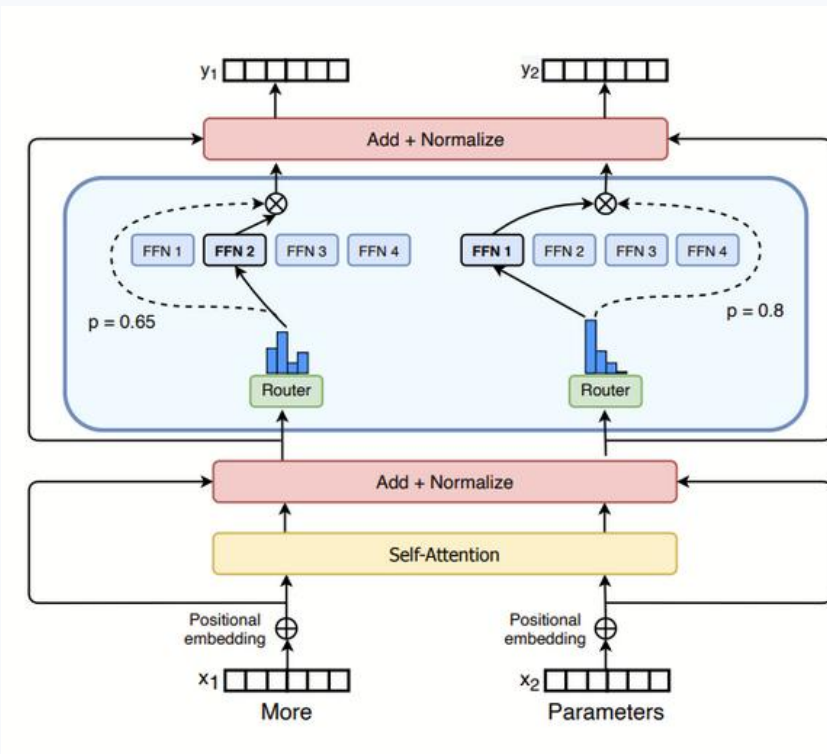
# Mixture of Experts (MoE)

<https://huggingface.co/blog/moe>

For each expert  $i$ :

- $$H(x)_i = (x \cdot W_g)_i + \mathcal{N}(0,1) \cdot \text{SoftPlus}(x \cdot W_{\text{noise}})_i,$$

where  $\text{SoftPlus}(z) = \log(1 + \exp(z))$



# Mixture of Experts (MoE)

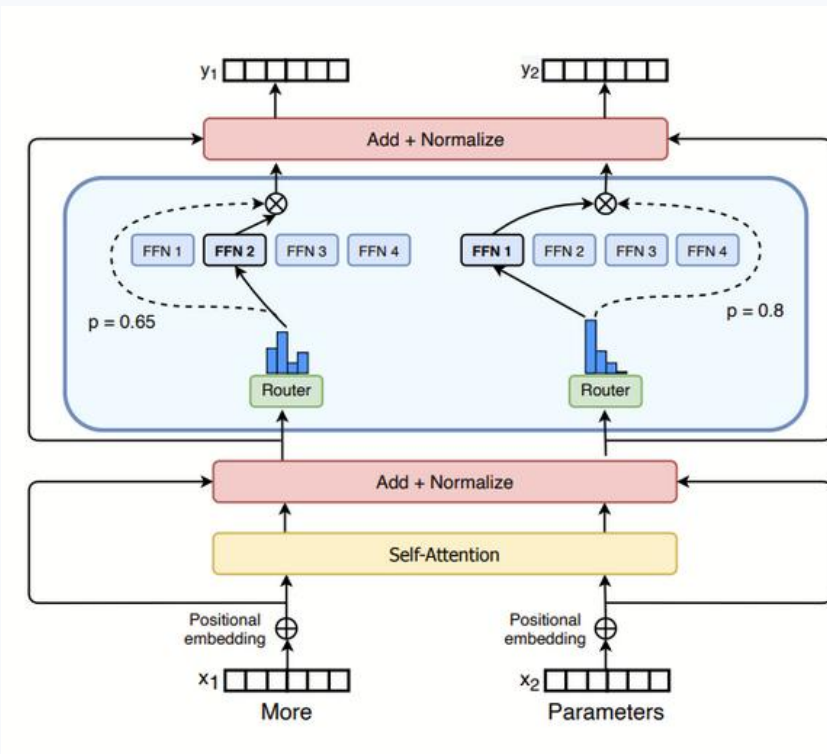
<https://huggingface.co/blog/moe>

For each expert  $i$ :

- $$H(x)_i = (x \cdot W_g)_i + \mathcal{N}(0,1) \cdot \text{SoftPlus}(x \cdot W_{\text{noise}})_i,$$

where  $\text{SoftPlus}(z) = \log(1 + \exp(z))$

- $$\text{KeepTopK}(H(x), k)_i = \begin{cases} H(x)_i, & \text{if } H(x)_i \text{ in top } k \text{ of } H(x) \\ -\infty, & \text{otherwise} \end{cases}$$



# Mixture of Experts (MoE)

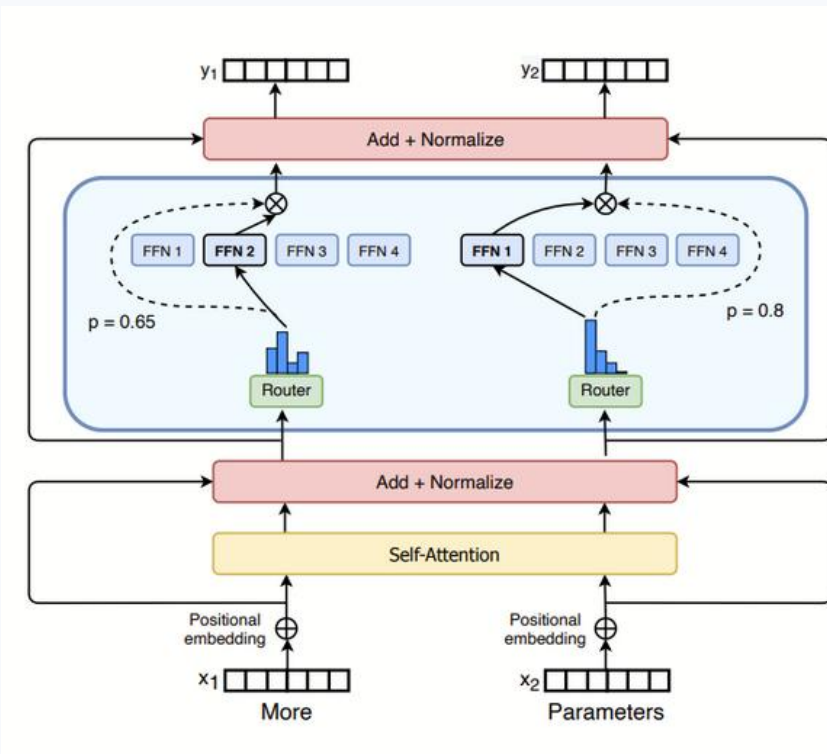
<https://huggingface.co/blog/moe>

For each expert  $i$ :

- $$H(x)_i = (x \cdot W_g)_i + \mathcal{N}(0,1) \cdot \text{SoftPlus}(x \cdot W_{\text{noise}})_i,$$

where  $\text{SoftPlus}(z) = \log(1 + \exp(z))$

- $$\text{KeepTopK}(H(x), k)_i = \begin{cases} H(x)_i, & \text{if } H(x)_i \text{ in top } k \text{ of } H(x) \\ -\infty, & \text{otherwise} \end{cases}$$
- $$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$



# Mixture of Experts (MoE)

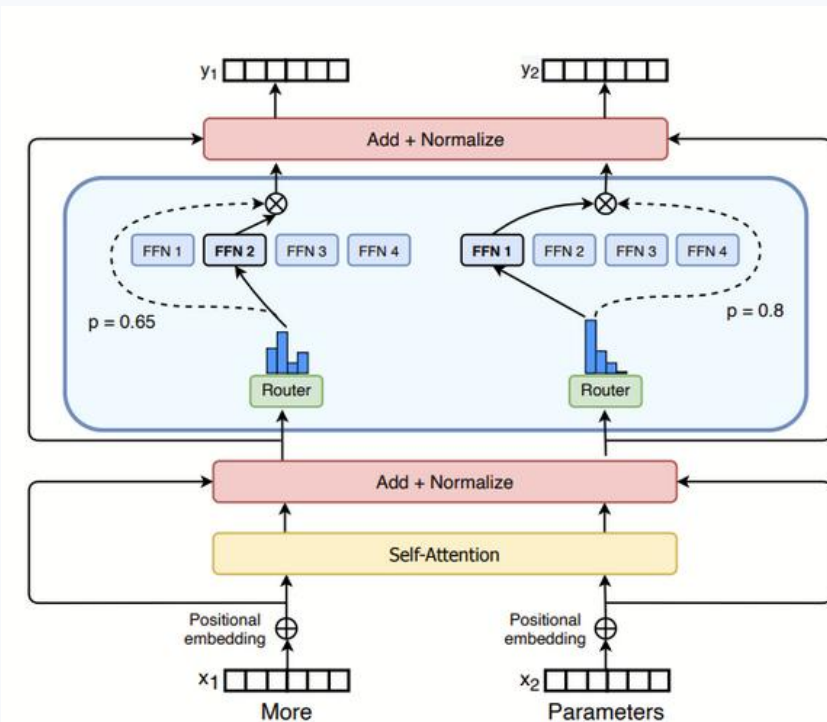
<https://huggingface.co/blog/moe>

For each expert  $i$ :

- $$H(x)_i = (x \cdot W_g)_i + \mathcal{N}(0,1) \cdot \text{SoftPlus}(x \cdot W_{\text{noise}})_i,$$

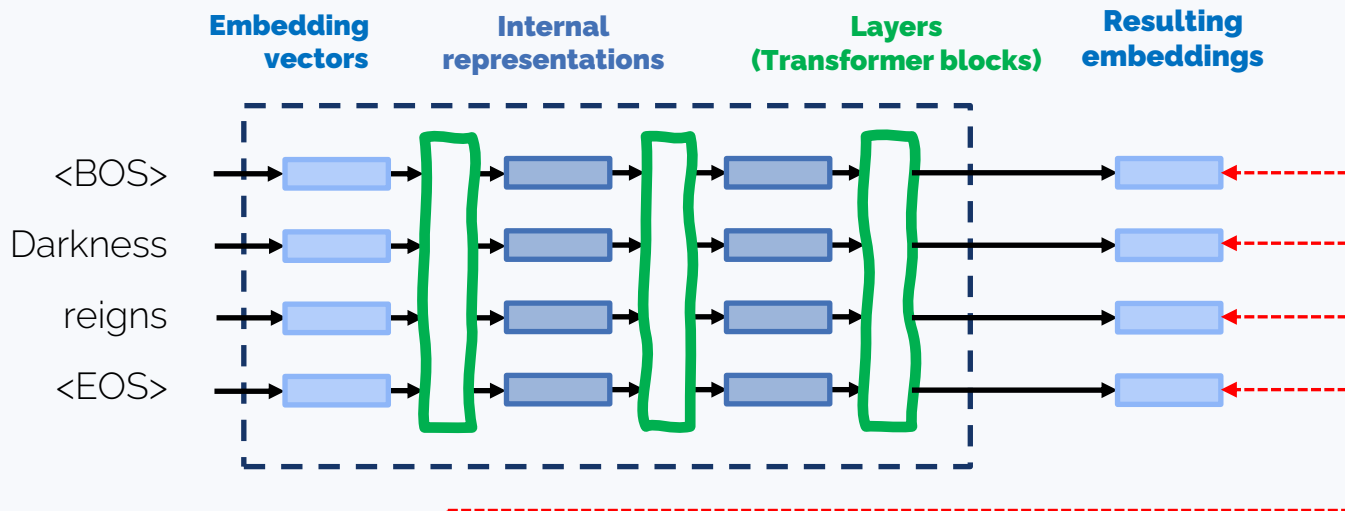
where  $\text{SoftPlus}(z) = \log(1 + \exp(z))$

- $$\text{KeepTopK}(H(x), k)_i = \begin{cases} H(x)_i, & \text{if } H(x)_i \text{ in top } k \text{ of } H(x) \\ -\infty, & \text{otherwise} \end{cases}$$
- $$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$
- $$y = \sum_i G(x)_i E_i(x)$$

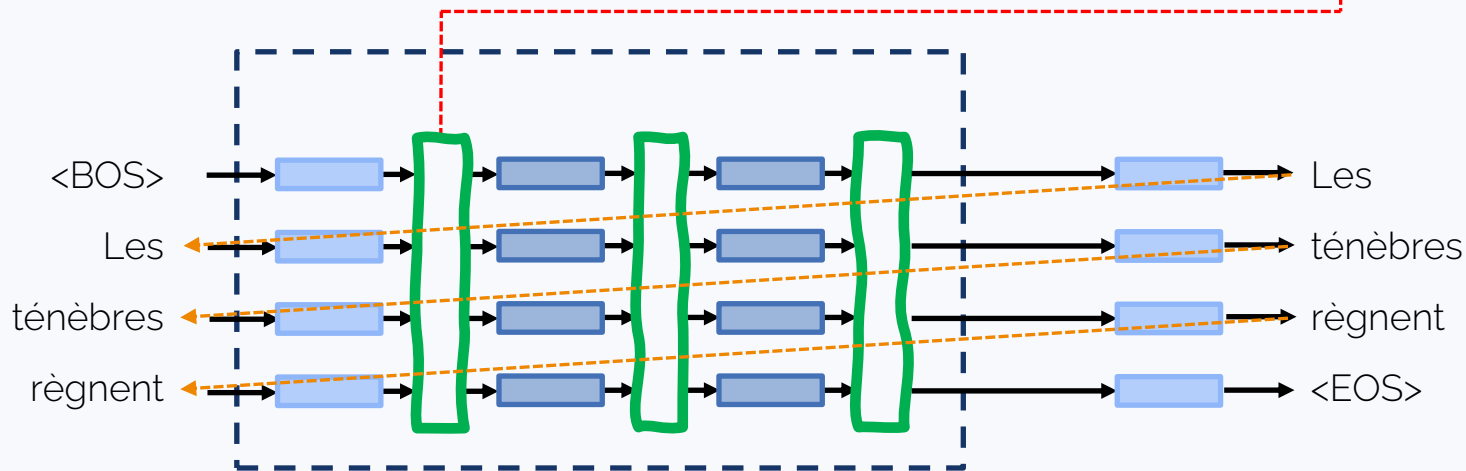


# Encoder-decoder transformers

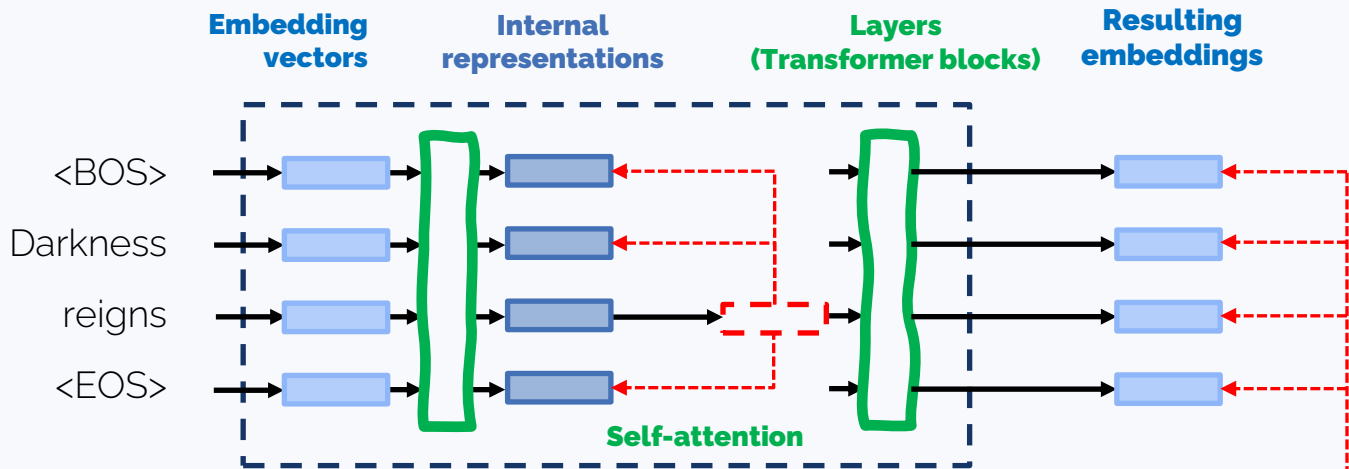
## Encoder



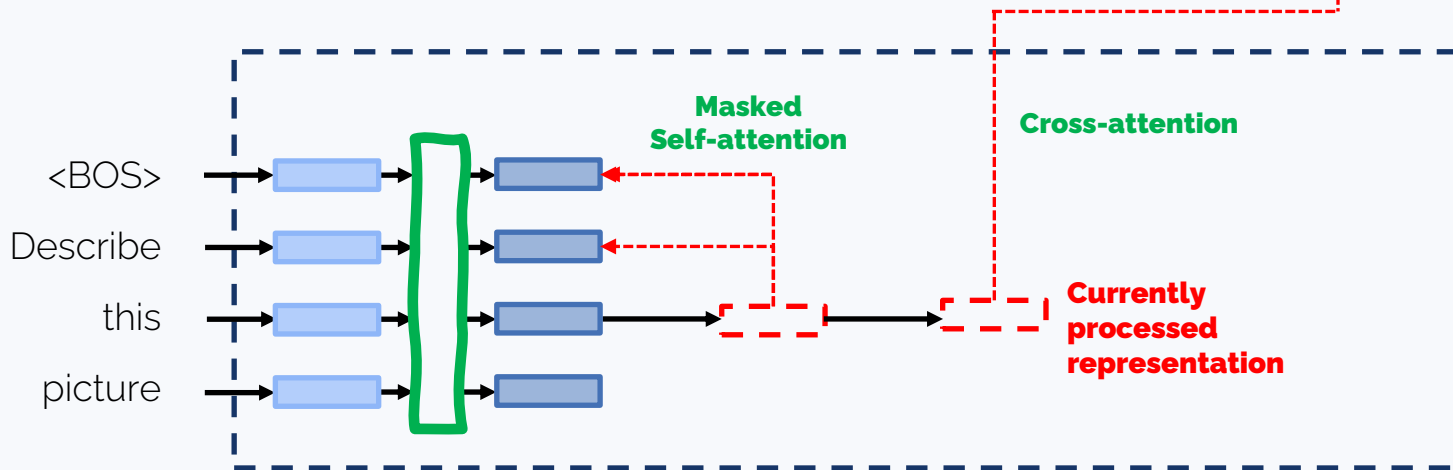
## Decoder



## Encoder



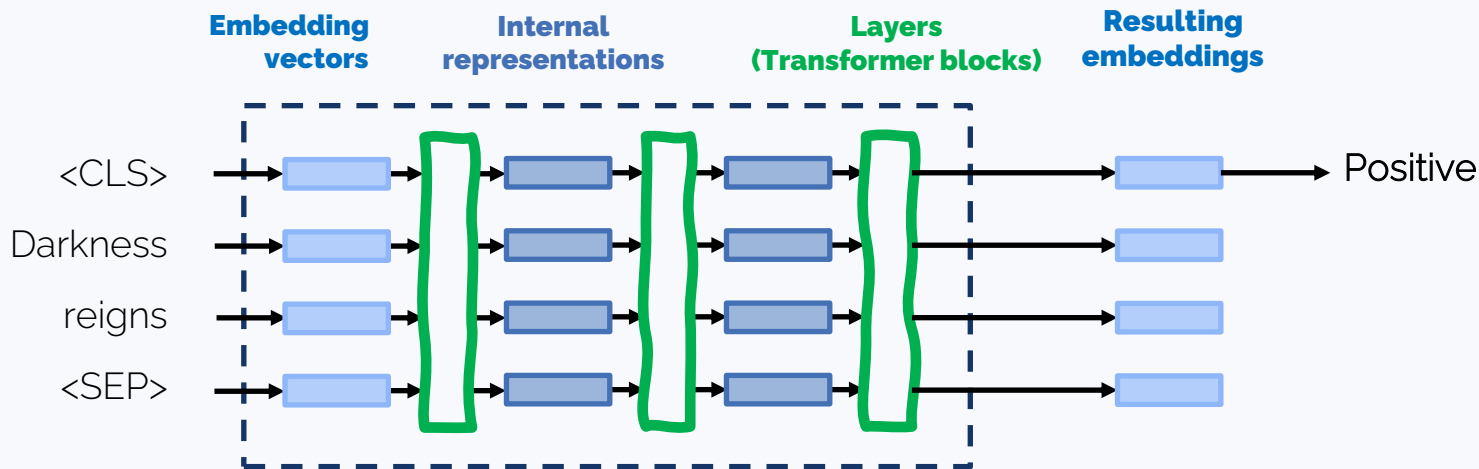
## Decoder



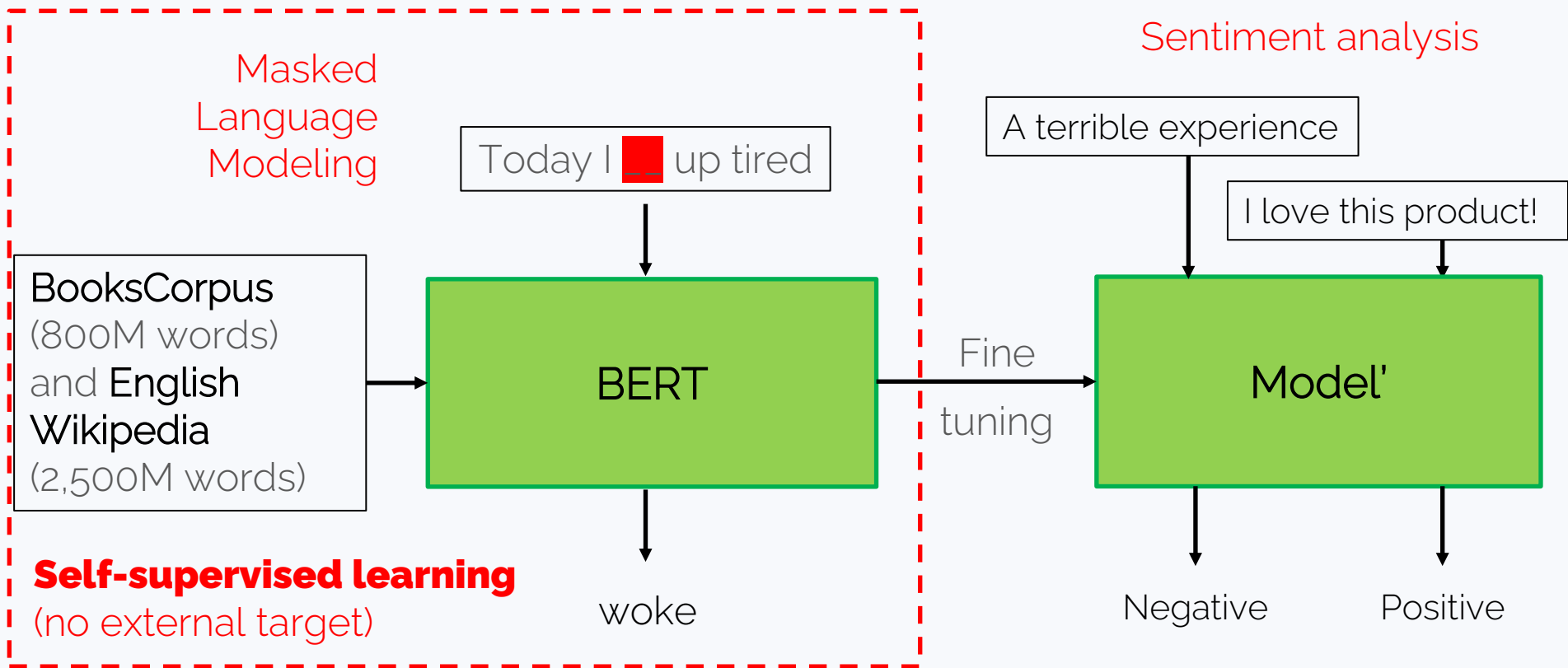


Encoder only  
transformers

## Encoder



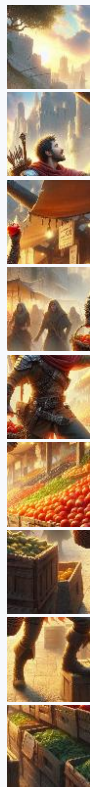
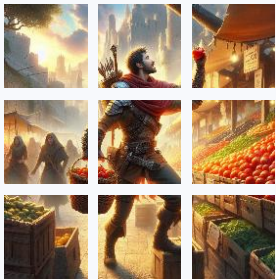
# Example: BERT



# Vision transformers



Breaking  
into  
patches

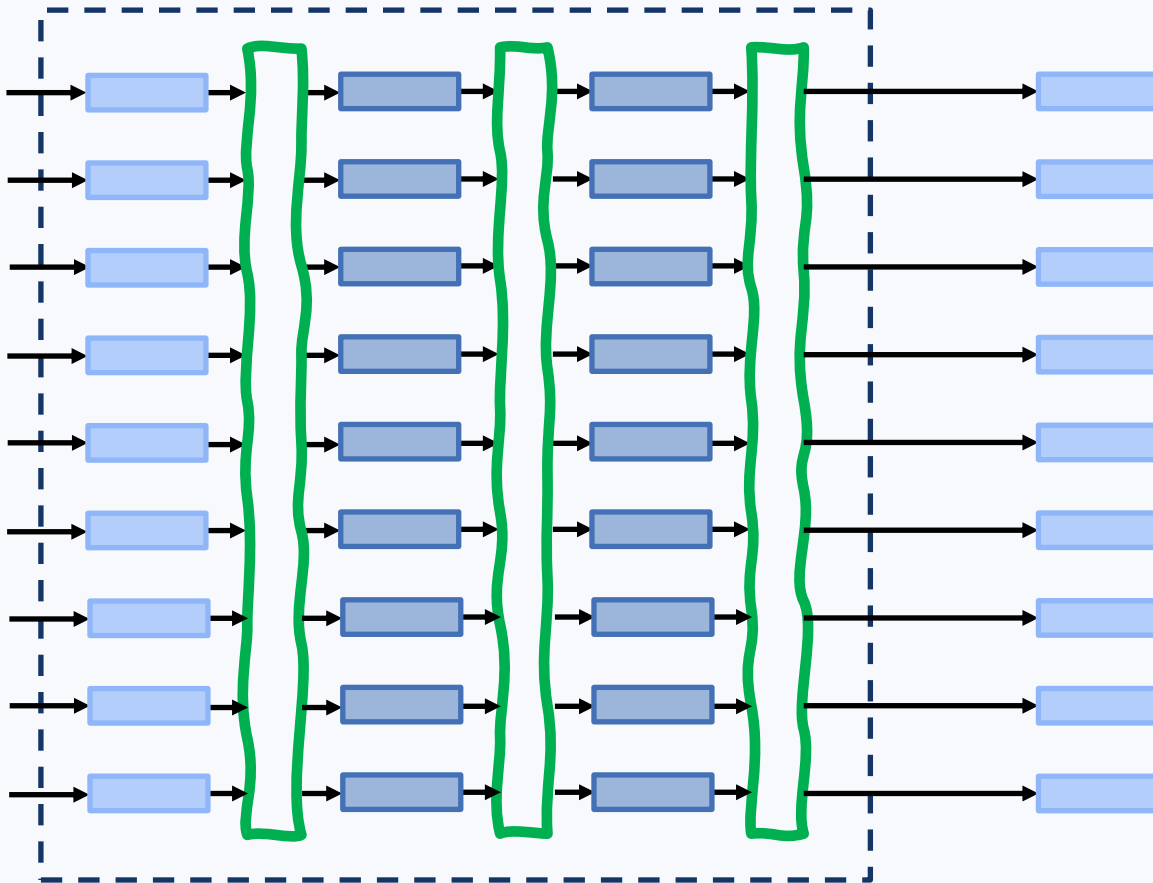


Embedding  
vectors

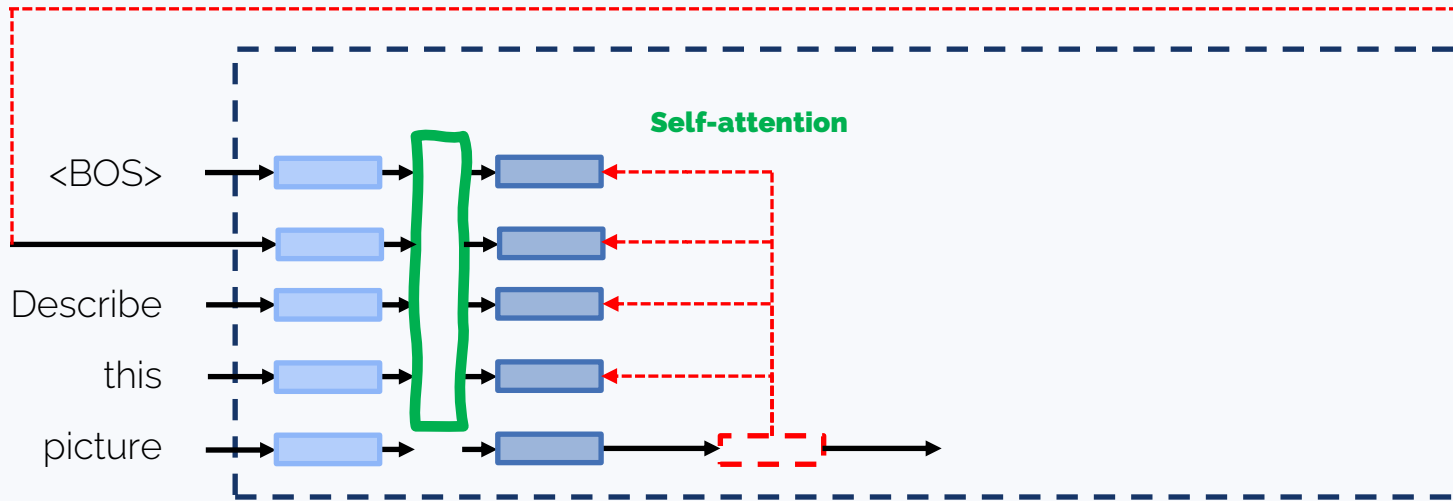
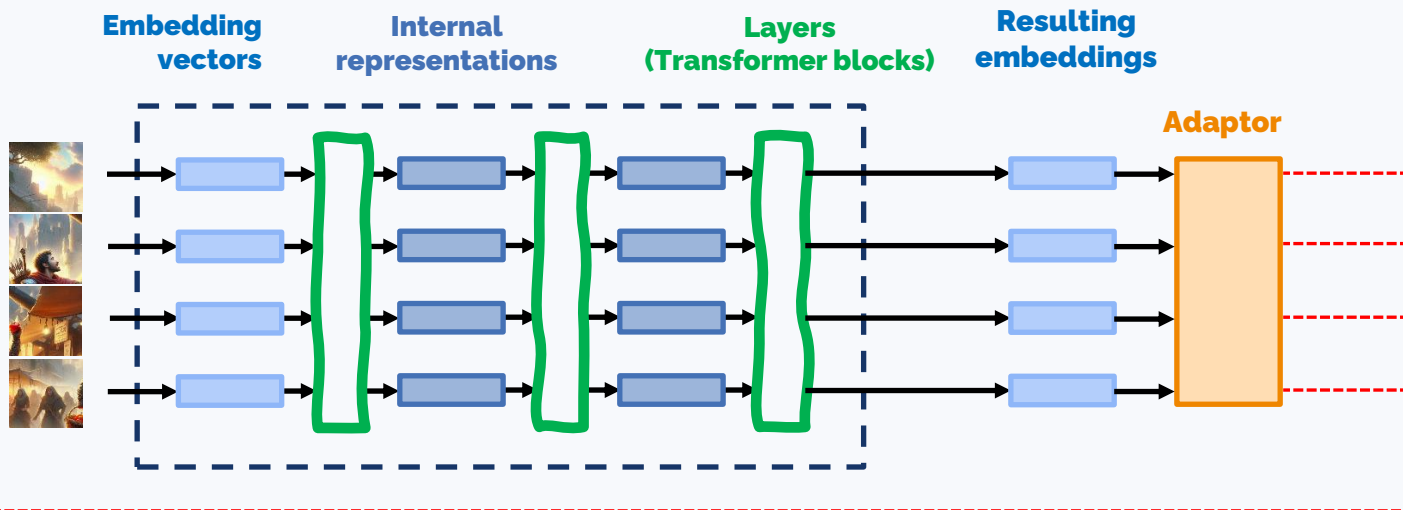
Internal  
representations

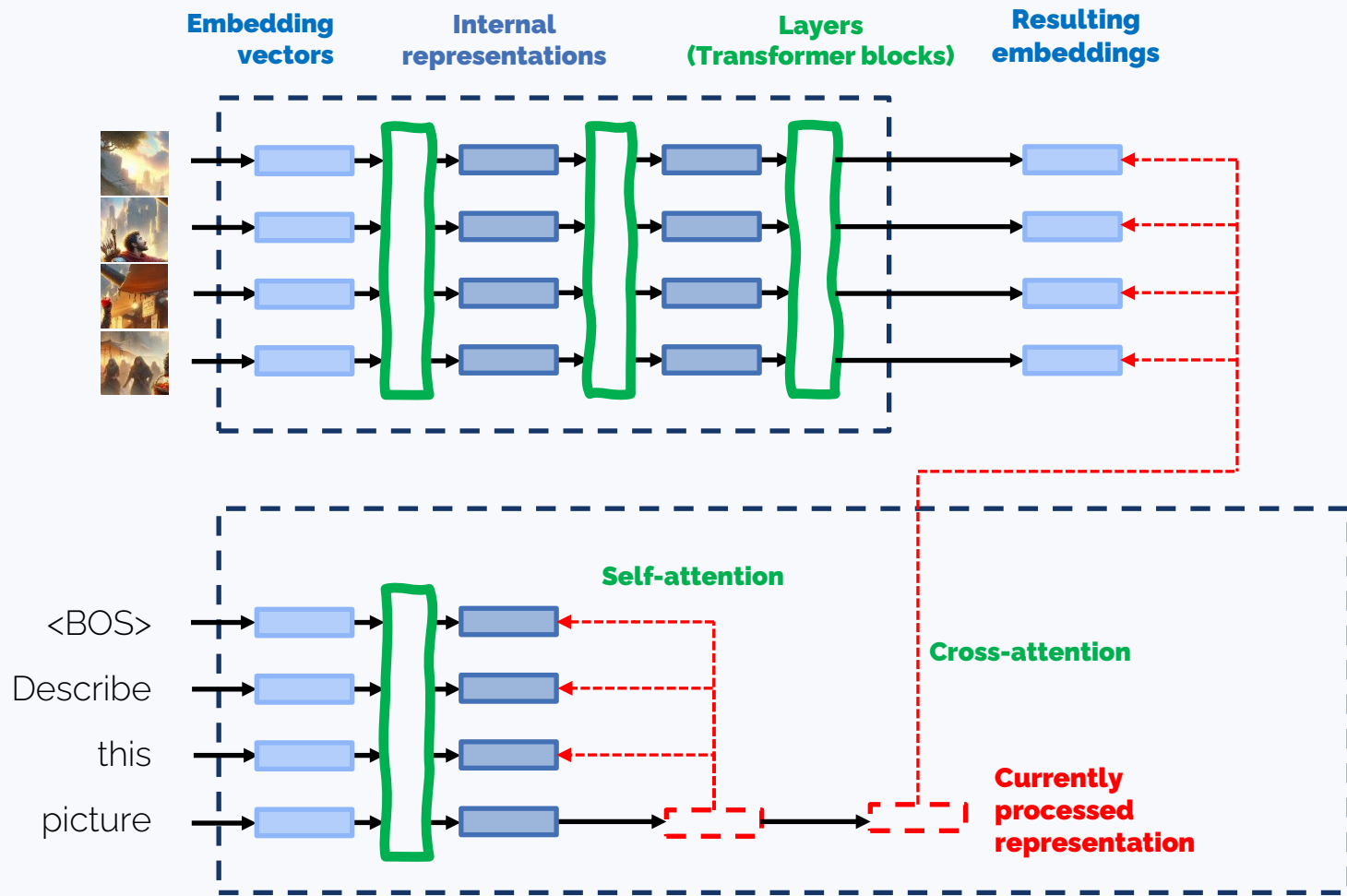
Layers  
(Transformer blocks)

Resulting  
embeddings



# Multimodality







Dissection time!

# Finding Global Lyapunov functions with transformers

# Lyapunov stability of dynamic systems

An equilibrium  $x^*$  of a dynamical system  $\dot{x} = f(x)$  is the point where  $f(x) = 0$ . Then,  $\dot{x} = 0$  and a trajectory never leaves this point.

An equilibrium is **Lyapunov stable** if  $\forall \varepsilon > 0, \exists \delta > 0$  such that whenever  $|x(0) - x^*| < \delta$ , the corresponding solution satisfies  $|x(t) - x^*| < \varepsilon, \forall t \geq 0$ .

That is: if you start sufficiently close to the equilibrium, you stay close forever.

# Lyapunov stability of dynamic systems

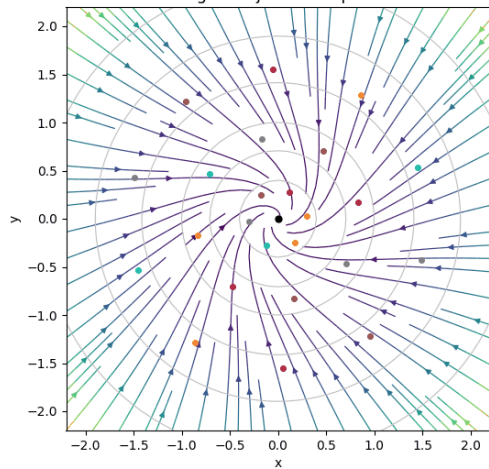
An equilibrium is **Lyapunov stable** if  $\forall \varepsilon > 0, \exists \delta > 0$  such that whenever  $|x(0) - x^*| < \delta$ , the corresponding solution satisfies  $|x(t) - x^*| < \varepsilon, \forall t \geq 0$ .

$$\begin{cases} \dot{x} = y - x - x \cdot (x^2 + y^2) \\ \dot{y} = -x - y - y \cdot (x^2 + y^2) \end{cases}$$

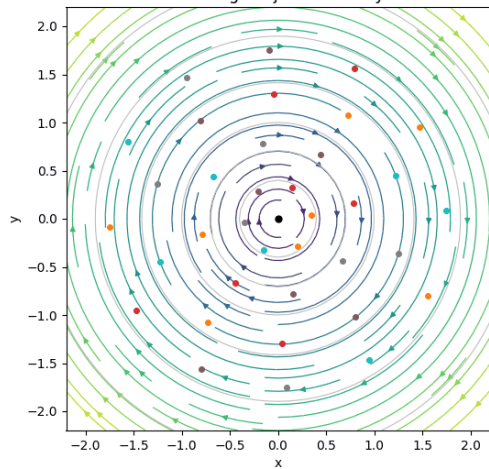
$$\begin{cases} \dot{x} = y \\ \dot{y} = -x \end{cases}$$

$$\begin{cases} \dot{x} = y + x - x \cdot (x^2 + y^2) \\ \dot{y} = -x + y - y \cdot (x^2 + y^2) \end{cases}$$

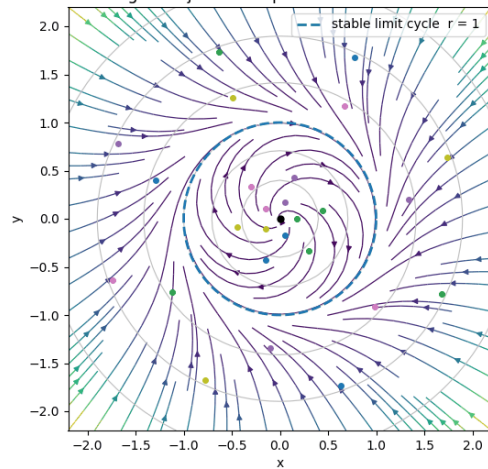
Stable origin: trajectories spiral inward



Stable but not attracting: trajectories stay on closed orbits



Unstable origin: trajectories spiral toward a stable limit cycle



Global Lyapunov functions: a long-standing open problem in mathematics, with symbolic transformers

<https://arxiv.org/abs/2410.08304>

# Lyapunov stability of dynamic systems

An equilibrium is **Lyapunov stable** if  $\forall \varepsilon > 0, \exists \delta > 0$  such that whenever  $|x(0) - x^*| < \delta$ , the corresponding solution satisfies  $|x(t) - x^*| < \varepsilon, \forall t \geq 0$ .

A **global Lyapunov function** is a  $C^1$  function  $V: \mathbb{R}^n \rightarrow \mathbb{R}_+$  such that

$$V(0) = 0, \quad V(x) > 0 \text{ for } x \neq 0, \quad \nabla V(x) \cdot f(x) \leq 0 \text{ for } x \neq 0$$

and  $V(x) \rightarrow +\infty$  as  $|x| \rightarrow \infty$ .

**Lyapunov's theorem:** if the system has such a Lyapunov function, then the equilibrium  $x = 0$  of  $\dot{x} = f(x)$  is stable.

**Intuition:** Along a trajectory,

$$\frac{d}{dt}V(x(t)) = \nabla V(x(t)) \cdot f(x(t)) \leq 0.$$

So trajectories are forced to move “downhill” or at least never uphill in  $V$ .

# Lyapunov stability of dynamic systems

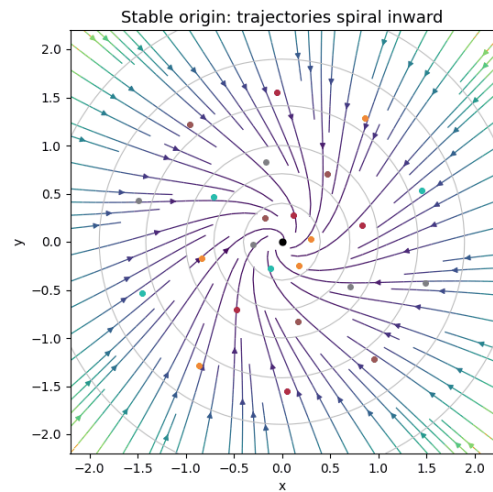
An equilibrium is **Lyapunov stable** if  $\forall \varepsilon > 0, \exists \delta > 0$  such that whenever  $|x(0) - x^*| < \delta$ , the corresponding solution satisfies  $|x(t) - x^*| < \varepsilon, \forall t \geq 0$ .

$$\begin{cases} \dot{x} = y - x - x \cdot (x^2 + y^2) \\ \dot{y} = -x - y - y \cdot (x^2 + y^2) \end{cases}$$

Take  $V(x, y) = \frac{1}{2}(x^2 + y^2)$ . Then

$$\begin{aligned} \dot{V} &= x\dot{x} + y\dot{y} \\ &= x(y - x - x \cdot (x^2 + y^2)) + y(-x - y - y \cdot (x^2 + y^2)) = \\ &= -(x^2 + y^2) - (x^2 + y^2)^2 < 0 \end{aligned}$$

So,  $V$  is a Lyapunov function.



Global Lyapunov functions: a long-standing open problem in mathematics, with symbolic transformers

<https://arxiv.org/abs/2410.08304>

# Lyapunov stability of dynamic systems

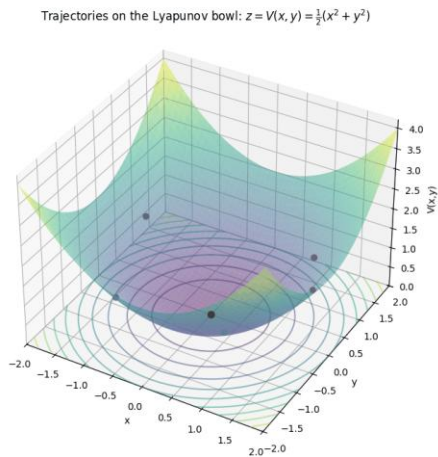
An equilibrium is **Lyapunov stable** if  $\forall \varepsilon > 0, \exists \delta > 0$  such that whenever  $|x(0) - x^*| < \delta$ , the corresponding solution satisfies  $|x(t) - x^*| < \varepsilon, \forall t \geq 0$ .

$$\begin{cases} \dot{x} = y - x - x \cdot (x^2 + y^2) \\ \dot{y} = -x - y - y \cdot (x^2 + y^2) \end{cases}$$

Take  $V(x, y) = \frac{1}{2}(x^2 + y^2)$ . Then

$$\begin{aligned} \dot{V} &= x\dot{x} + y\dot{y} \\ &= x(y - x - x \cdot (x^2 + y^2)) + y(-x - y - y \cdot (x^2 + y^2)) = \\ &= -(x^2 + y^2) - (x^2 + y^2)^2 < 0 \end{aligned}$$

So,  $V$  is a Lyapunov function.



Global Lyapunov functions: a long-standing open problem in mathematics, with symbolic transformers

<https://arxiv.org/abs/2410.08304>

# Where to get data

A **global Lyapunov function** is a  $\mathcal{C}^1$  function  $V: \mathbb{R}^n \rightarrow \mathbb{R}_+$  such that

$$V(0) = 0, \quad V(x) > 0 \text{ for } x \neq 0, \quad \nabla V(x) \cdot f(x) \leq 0 \text{ for } x \neq 0$$

and  $V(x) \rightarrow +\infty$  as  $|x| \rightarrow \infty$ .

## Backward generation:

- Start with  $V_{proper}$  with global min at  $0$  and  $V_{cross} \geq 0$ . Take  $V = V_{proper} + V_{cross}$
- Sample functions  $e_i(x)$  from the orthogonal complement to  $\nabla V$
- Take some  $h_i(x)$  (some of them zero)
- Build

$$f(x) = -(h_1(x)\nabla V(x)_1, \dots, h_d(x)\nabla V(x)_d) + \sum_i g_i(x)e_i(x)$$

- Simplify the functions  $f_i$ , obscuring patterns from the generative process

## Forward generation:

- Generate a polynomial system at random
- Use a routine to find a polynomial sum-of-squares (SOS) Lyapunov function.
- Keep the system if such function exists, restart from step 1 otherwise



# Tokenization

They use **Polish (pre-order) notation**:

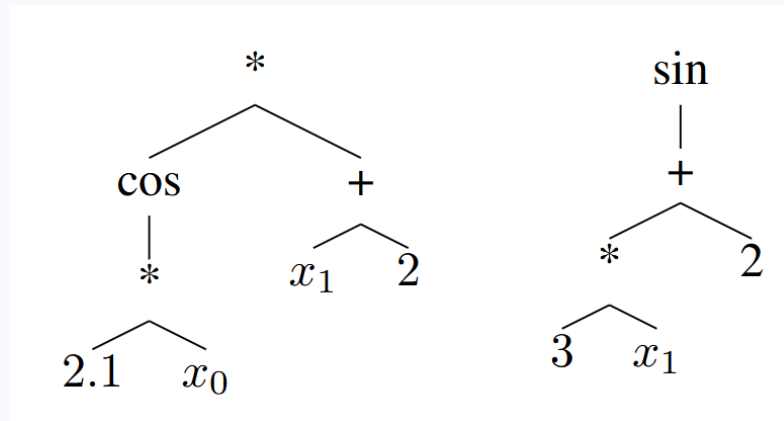
$$\begin{cases} \dot{x}_0 = \cos(2.1x)(x_1 + 2) \\ \dot{x}_1 = \sin(3x_1 + 2) \end{cases}$$

Write each as:

- `[*, cos, *, 2.1, x0, +, x1, 2]` and
- `[sin, +, *, 3, x1, 2]`.

Finally:

`[*, cos, *, 2.1, 10^-, 1, x0, +, x1, 2, SEP, sin, +, *, 3, x1, 2]`



# Model and training

They train transformers with

- **8** layers,
- **10** attention heads and
- an embedding dimension of **640**

on batches of **16** examples, using the **Adam** optimizer with a **lr=10<sup>-4</sup>**,

- an initial linear warm-up phase of 10,000 optimization steps (**lr** grows linearly from 0),
- and inverse square root scheduling ( $lr(t) \propto \frac{1}{\sqrt{t}}$ ).

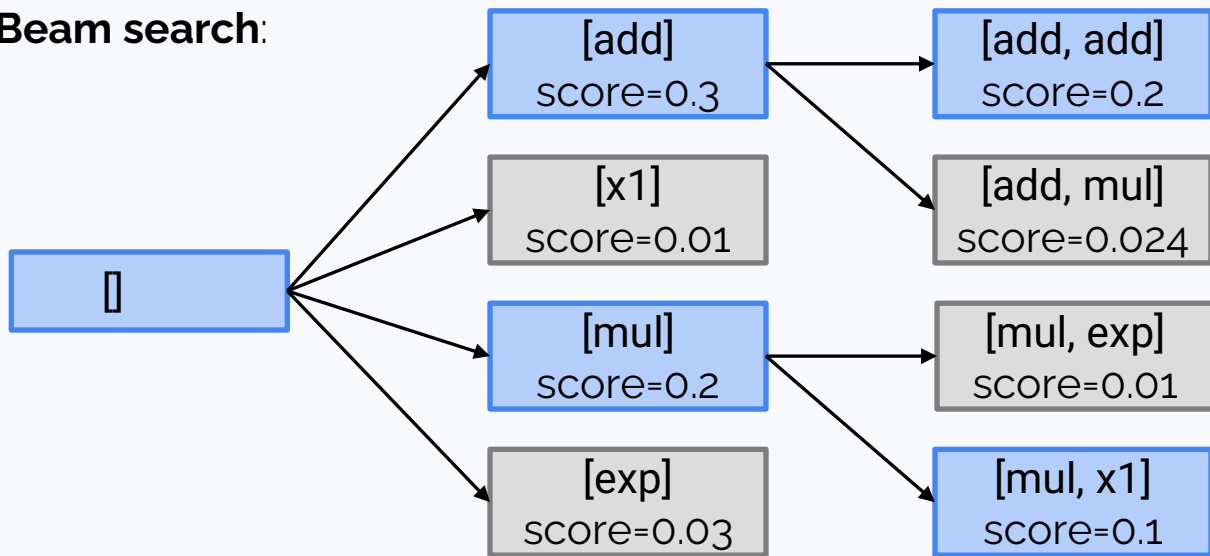
All experiments run on 8 V100 GPU with 32 GB of memory, for 3 or 4 epochs of 2.4M examples/epoch. Training time is between 12 to 15 hours per GPU.

# Results

| Backward datasets   | Accuracy |       |  | Forward datasets | Accuracy |       |
|---------------------|----------|-------|--|------------------|----------|-------|
|                     | bs=1     | bs=50 |  |                  | bs=1     | bs=50 |
| BPoly (polynomial)  | 99       | 100   |  | FBarr (barrier)  | 93       | 98    |
| BNonPoly (non-poly) | 77       | 87    |  | FLyap (Lyapunov) | 81       | 88    |

Table 2: **In-domain accuracy of models.** Beam size (bs) 1 and 50.

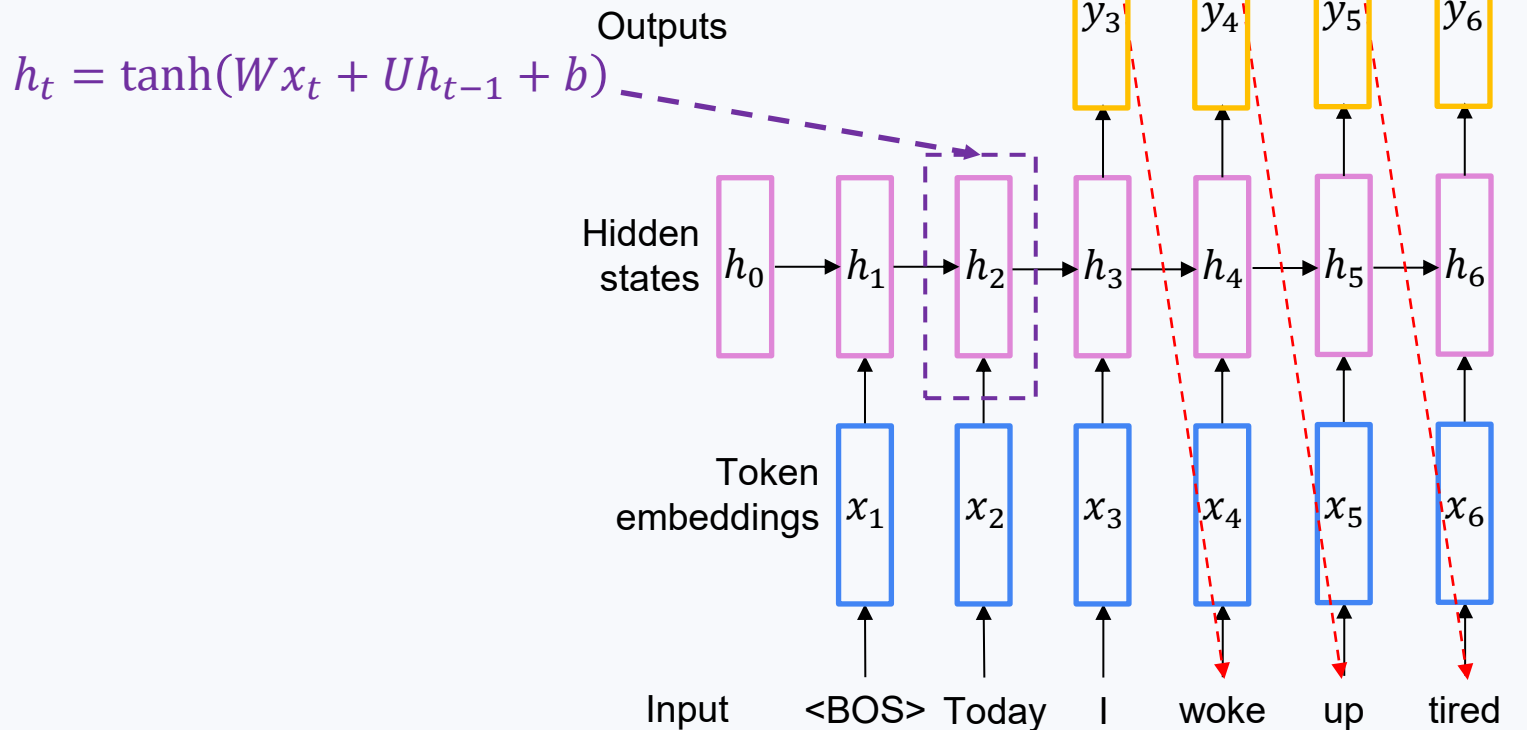
Beam search:



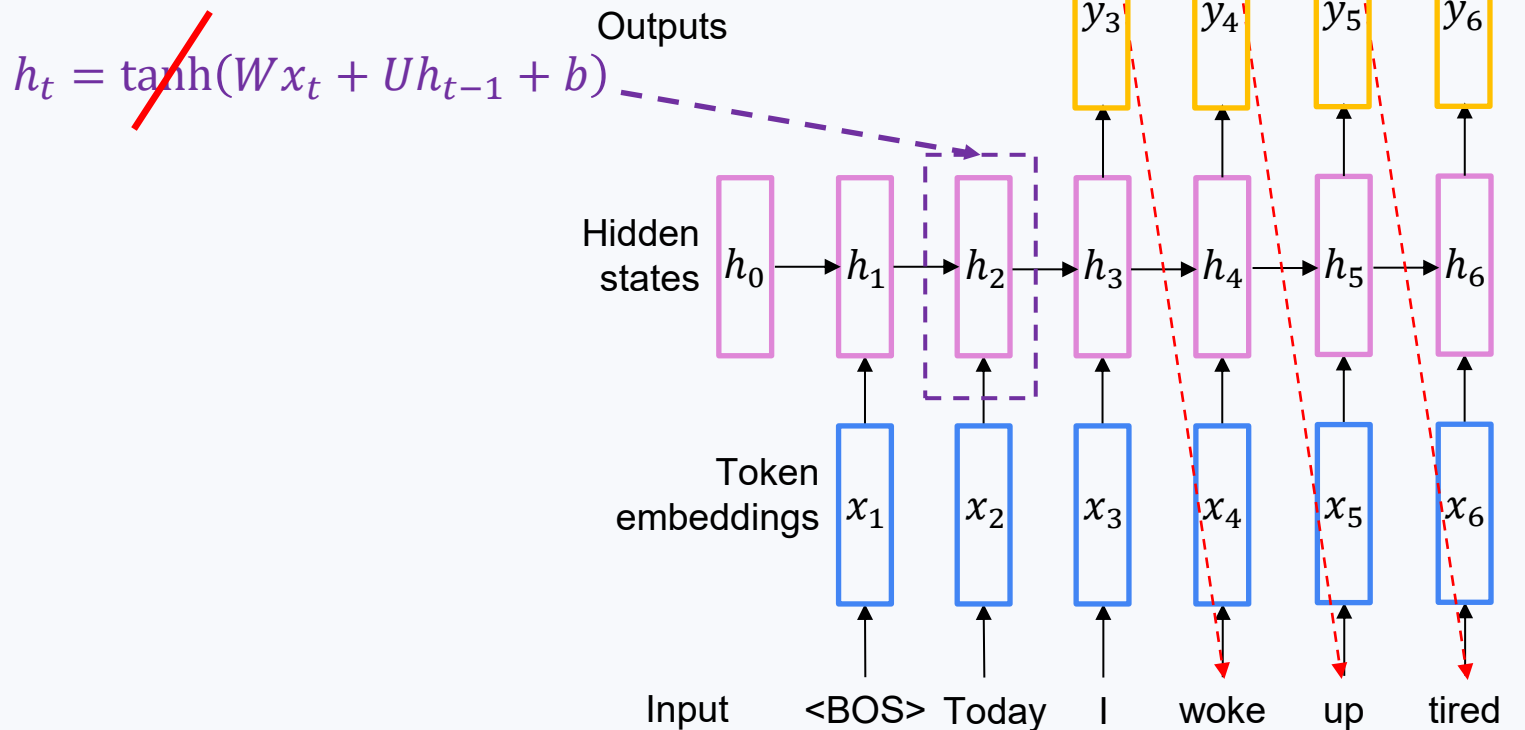
Training time!

# Transformer alternatives

# Language modelling with RNNs

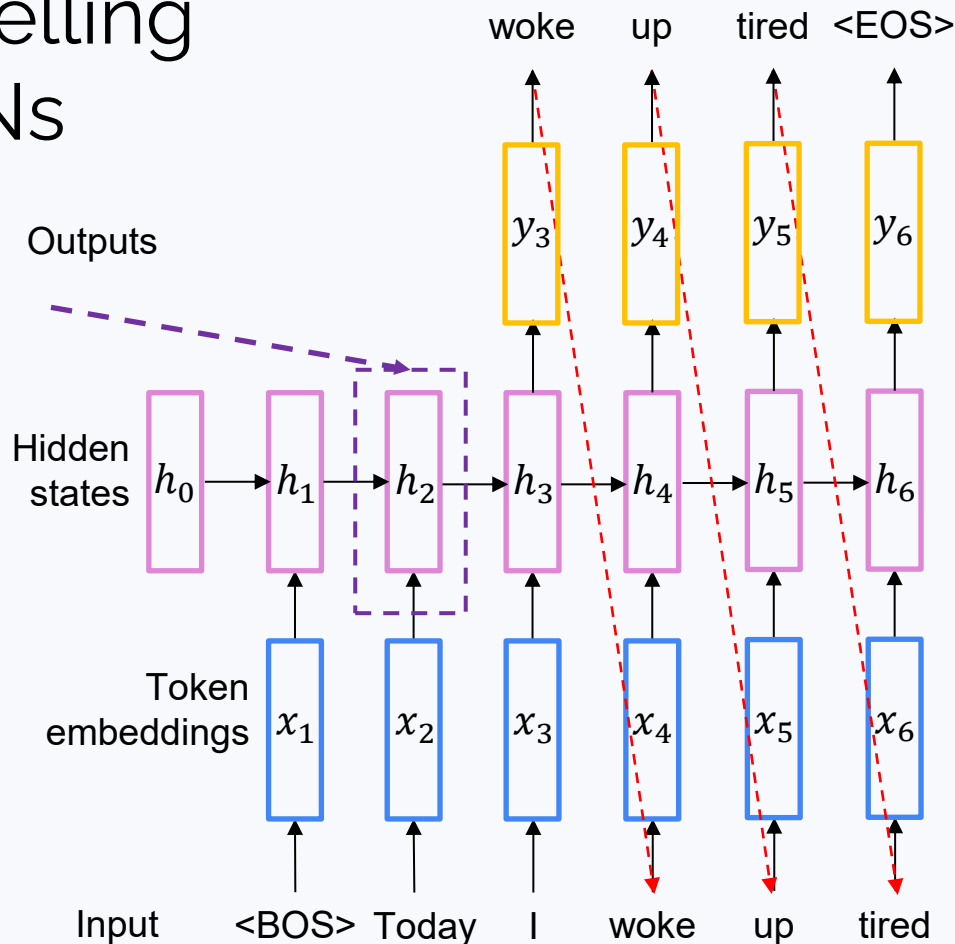


# Language modelling with **Linear** RNNs



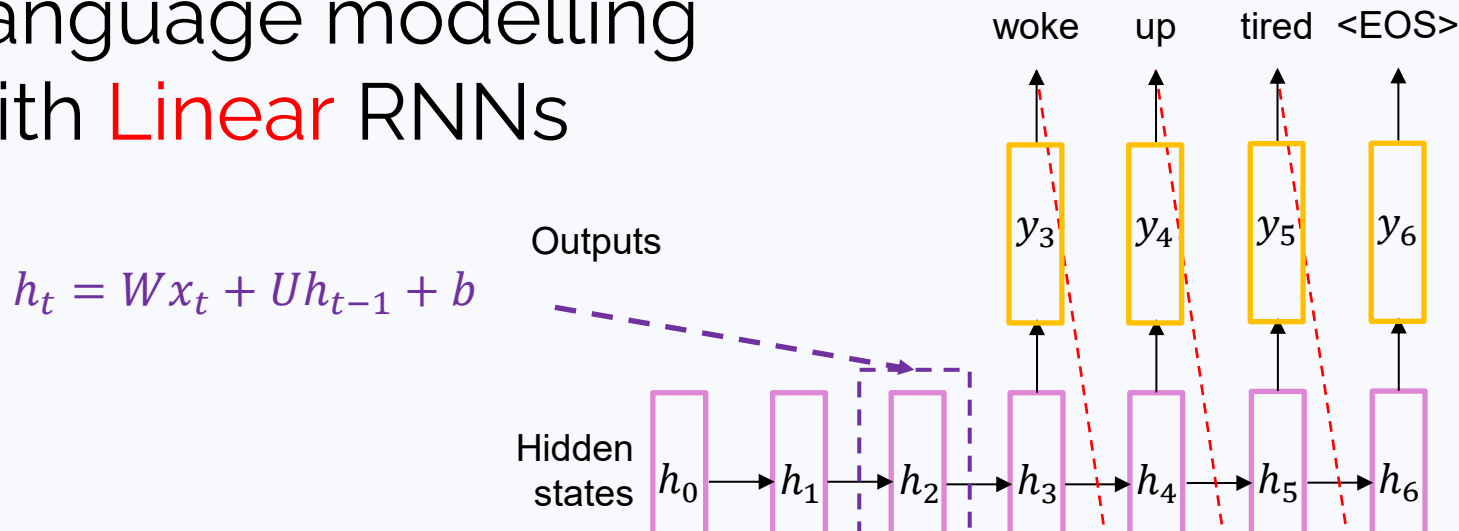
# Language modelling with **Linear** RNNs

$$h_t = Wx_t + Uh_{t-1} + b$$





# Language modelling with **Linear** RNNs

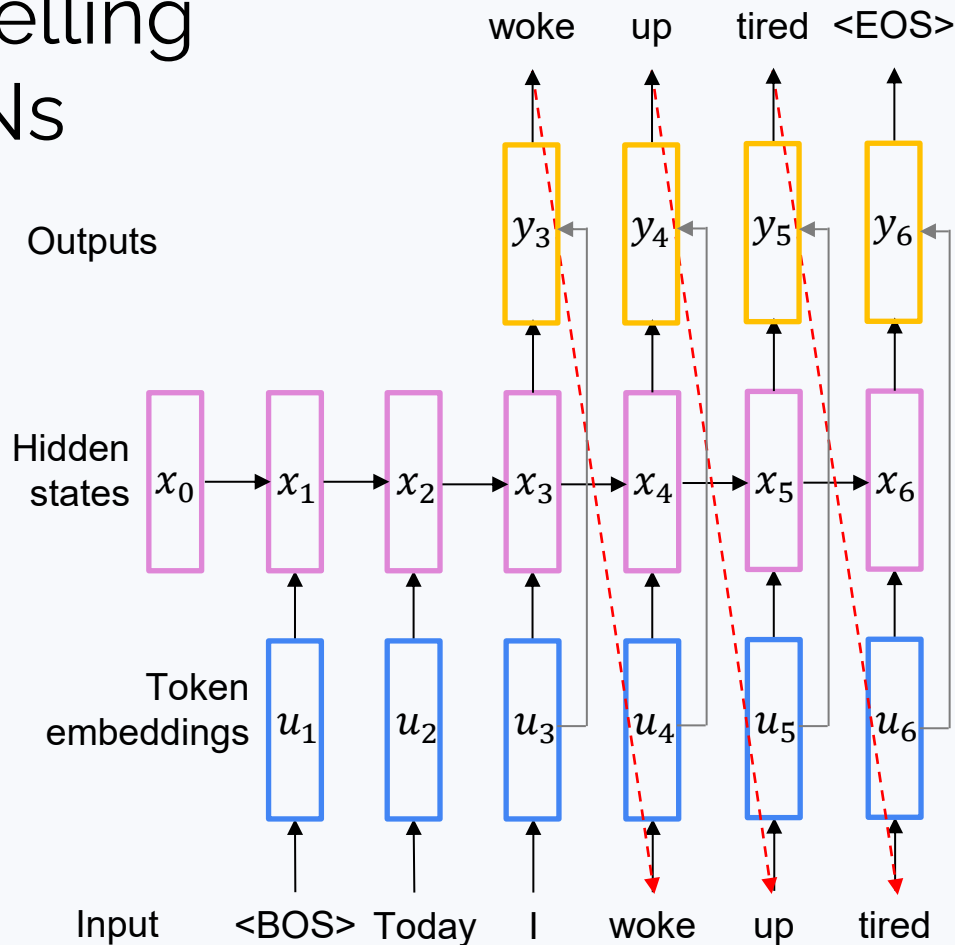


| RECURRENCE | sCIFAR            | LISTOPS           | TEXT              | RETRIEVAL         |
|------------|-------------------|-------------------|-------------------|-------------------|
| RNN-RELU   | 69.7 (0.2)        | 37.6 (8.0)        | 88.0 (0.1)        | 88.5 (0.1)        |
| RNN-TANH   | 69.9 (0.3)        | 43.9 (0.1)        | 87.2 (0.1)        | 88.9 (0.2)        |
| RNN-LIN    | <b>72.2</b> (0.2) | <b>50.4</b> (0.2) | <b>89.1</b> (0.1) | <b>89.1</b> (0.1) |

Table 1 | *The effect of removing the nonlinearity from the recurrent unit on test accuracy (§3.1). We show here results only for the sCIFAR, ListOps, Text and Retrieval tasks in LRA as these models did not exceed random guessing on PathFinder/PathX (further improvements in Tb.2 and 3). Performance of deep SSMs shown in Tb.3.*

# Language modelling with **Linear** RNNs

$$x_k = Ax_{k-1} + Bu_k,$$
$$y_k = Cx_k + Du_k.$$



# Language modelling with Linear RNNs

$$\begin{aligned}x_k &= Ax_{k-1} + Bu_k, \\y_k &= Cx_k + Du_k.\end{aligned}$$

We can rewrite this as:

$$\begin{aligned}y_k &= CA^k Bu_0 + CA^{k-1} Bu_1 + \dots + CABu_{k-1} + CBu_k + Du_k = \\&= \mathcal{K}_L(A, B, C) * u + Du_k\end{aligned}$$

Where  $*$  is convolution with kernel

$$\mathcal{K}_L(A, B, C) = [CA^k B, CA^{k-1} B, \dots, CAB, CB]$$

# First improvements

$$x_k = Ax_{k-1} + Bu_k,$$

$$y_k = Cx_k + Du_k.$$

$$\begin{aligned} y_k &= CA^k Bu_0 + CA^{k-1} Bu_1 + \dots + CABu_{k-1} + CBu_k + Du_k = \\ &= \mathcal{K}_L(A, B, C) * u + Du_k \end{aligned}$$

1)  $A$  is better be simple – usually non-trainable, often diagonal + low-rank

$$\mathcal{K}_L(A, B, C) = [CA^k B, CA^{k-1} B, \dots, CAB, CB]$$

2) The convolution theorem:

$$g * h = \mathcal{F}^{-1}(\mathcal{F}(g) \odot \mathcal{F}(h))$$

This allows to reduce complexity from  $O(L^2)$  to  $O(L \log L)$ .

# Continuous-time formulation

$$\dot{x} = Ax + Bu,$$

$$y = Cx + Du.$$

By putting  $x_k = x(kT)$ , we can integrate and discretize it:

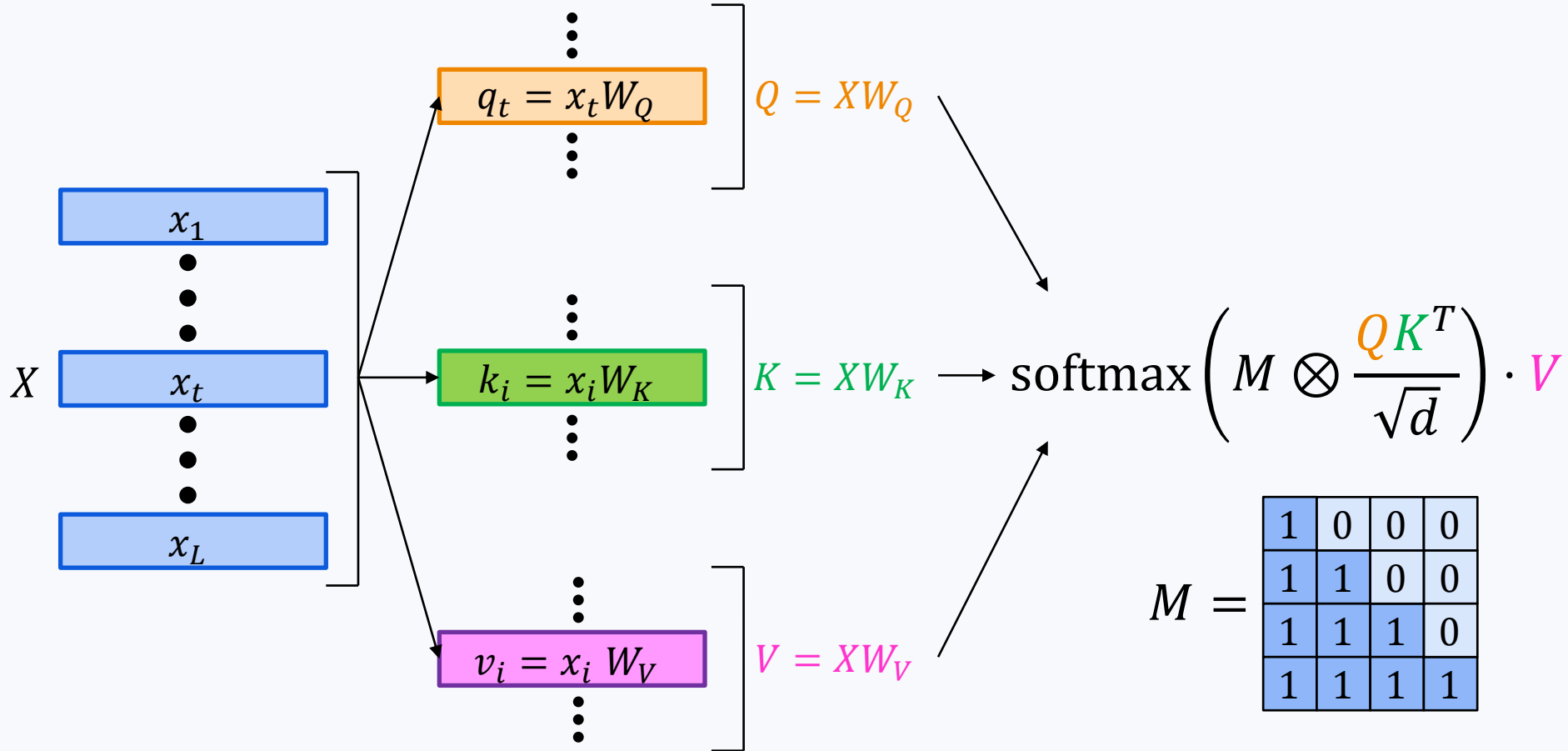
$$x_k = \bar{A}x_{k-1} + \bar{B}u_k,$$

$$y_k = Cx_k + Du_k.$$

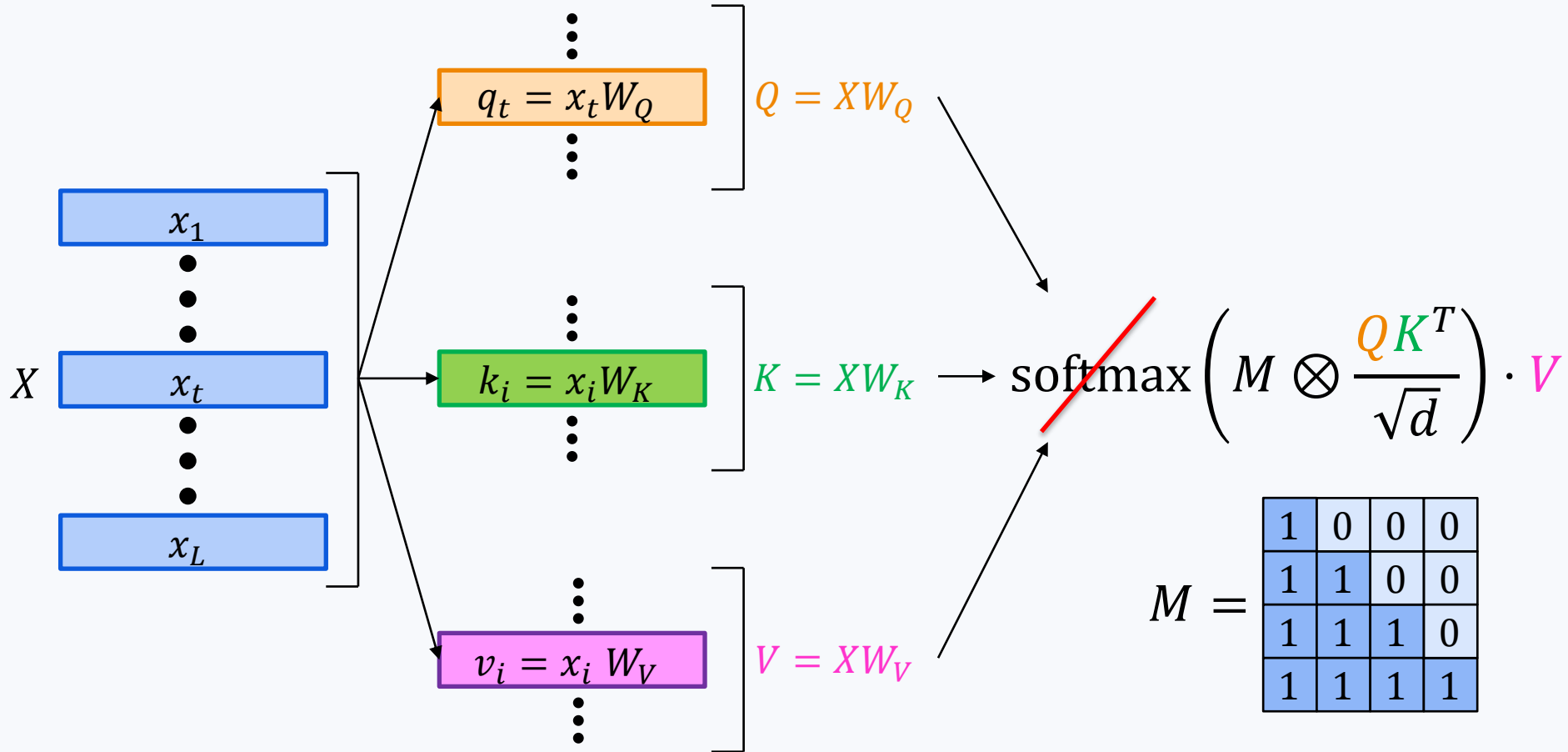
with

$$\bar{A} = e^{AT}, \quad \bar{B} = A^{-1}(e^{AT} - I)$$

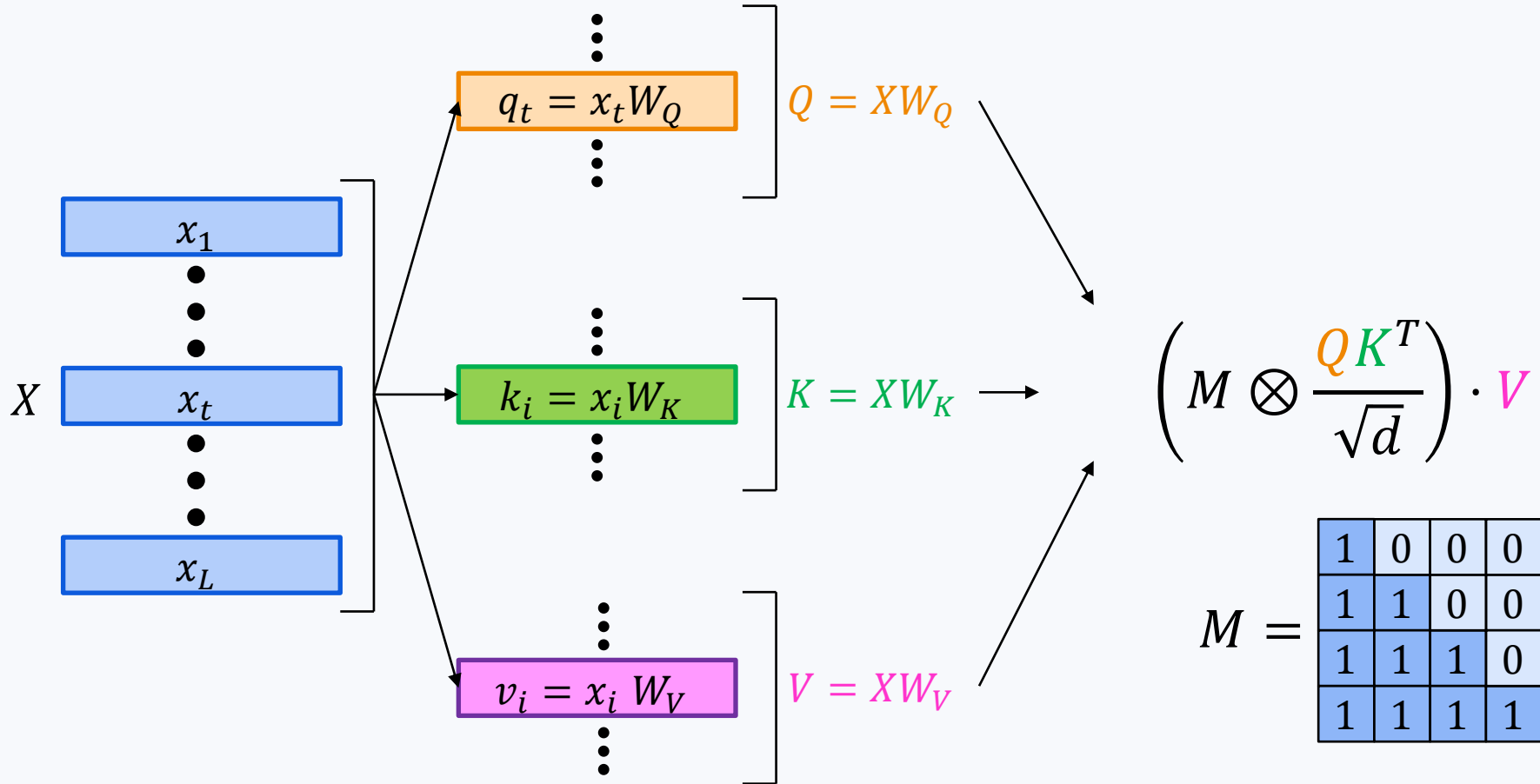
# Transformers are RNNs



# Transformers are RNNs



# Transformers are RNNs

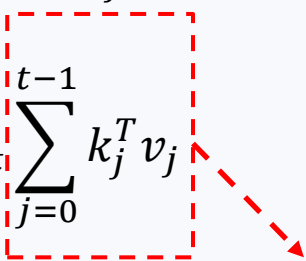




# Transformers are RNNs

$$\begin{aligned}\dot{x} &= Ax + Bu, \\ y &= Cx + Du.\end{aligned}$$

$$y_t = \sum_{j=0}^{t-1} (q_t k_j^T) v_j = \sum_{j=0}^{t-1} q_t (k_j^T v_j) =$$

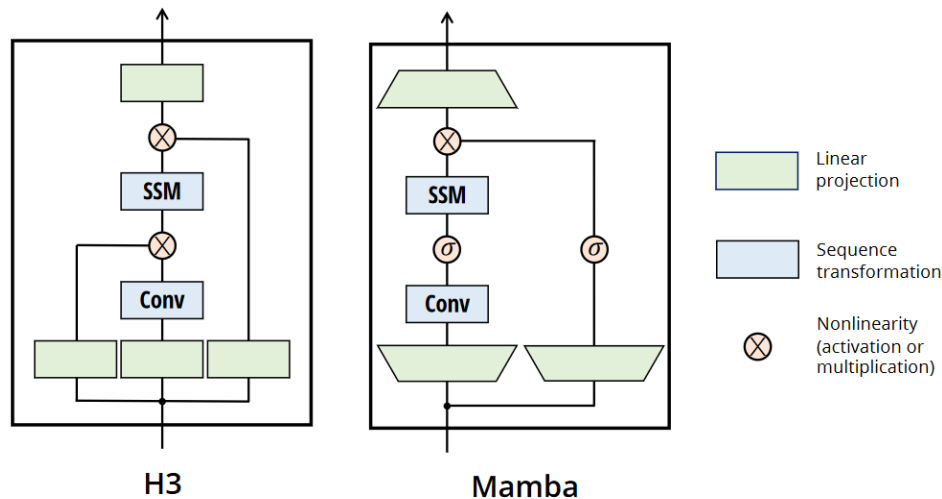
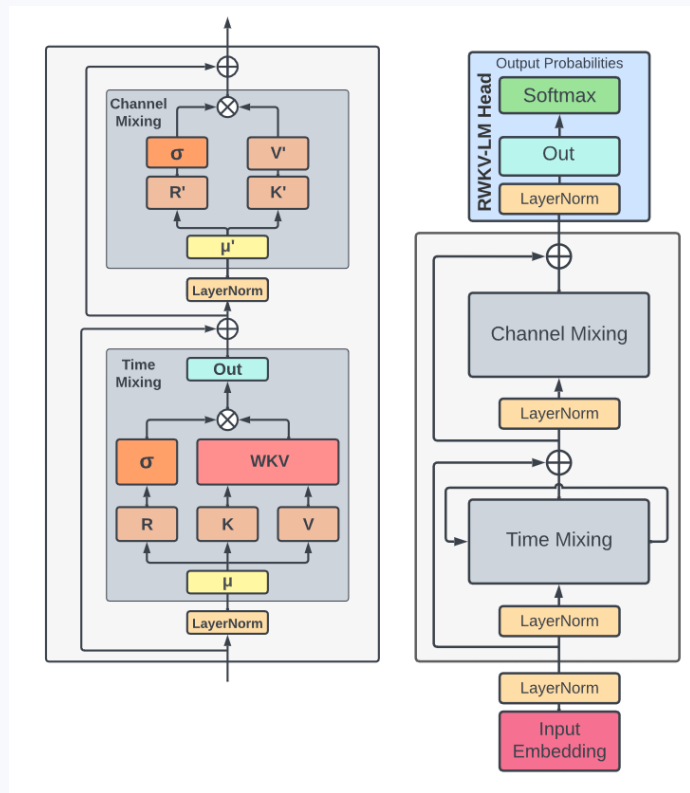
$$= q_t \sum_{j=0}^{t-1} k_j^T v_j$$


Hidden state of shape  $d \times d$ ,  
which is updated recurrently

$$\left( M \otimes \frac{QK^T}{\sqrt{d}} \right) \cdot V$$

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

# All in: Linear RNNs, QKV, convolutions & gating



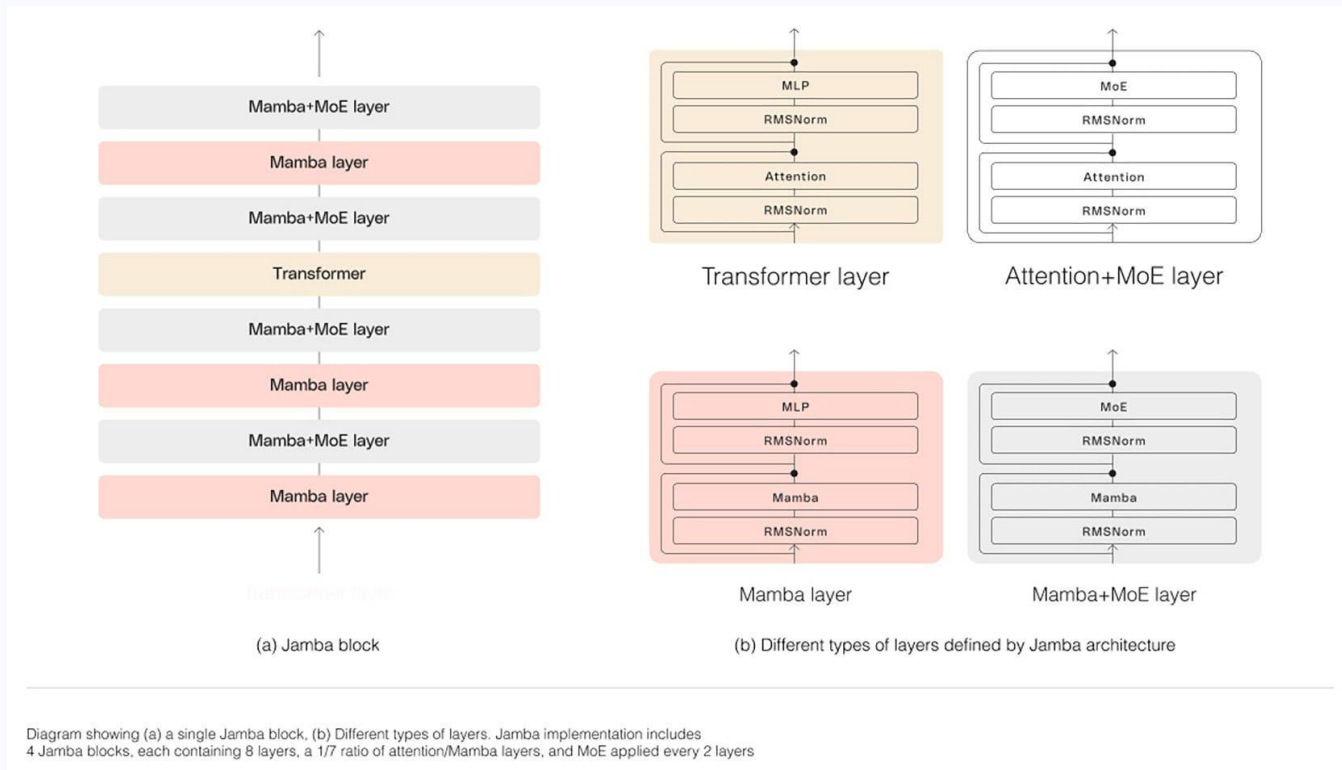
**Mamba** has also data-dependent discretization step  $T$  in

$$\bar{A} = e^{AT}, \quad \bar{B} = A^{-1}(e^{AT} - I)$$

RWKV: Reinventing RNNs for the Transformer Era <https://arxiv.org/abs/2305.13048>

Mamba: Linear-Time Sequence Modeling with Selective State Spaces <https://arxiv.org/abs/2312.00752>

# But... it's still not enough $\Rightarrow$ hybrid architectures



But... it's still not enough  $\Rightarrow$  hybrid architectures

